

Agent Plasticity: Measuring Self-Improvement Through Experience

Harman Singh^{1,2}, Anton Bakhtin², Rulin Shao^{2,3}, Gabriel Synnaeve², Ilya Kulikov², Rob Fergus², Sanjeev Arora⁴, Kurt Keutzer¹, Jason Weston², Anuj Mahajan^{2,†}, Anirudh Goyal^{2,†}

¹UC Berkeley, ²Meta Superintelligence Labs, ³University of Washington, ⁴Princeton University

AI agents increasingly operate in environments where they can diagnose failures and improve through experience, yet existing evaluations largely measure what an agent can do at a fixed point in time rather than how effectively it learns. Evaluating self-improvement requires answering three questions: does future performance improve and generalize beyond the interactions that enabled learning; how efficiently are new capabilities acquired; and where does the self-improvement process break down? To answer these questions, we study self-improvement in a controlled setting where agents amortize past experience into reusable artifacts that are inherited by future instances. At each checkpoint, we measure performance on training and held-out environment interactions while accounting for learning cost. We introduce *agent plasticity*, the efficiency with which an agent converts experience into gains in future held-out performance. Across multiple environments, frontier models exhibit sharply different improvement trajectories despite comparable opportunities to learn. Some achieve substantial and persistent gains, while others remain near or below their initial performance, and gains within the training regime often transfer only partially to out-of-distribution conditions. Endpoint capability and acquisition efficiency also diverge: the agent that ultimately performs best need not be the one that improves most efficiently. Tracing failures through the improvement loop further reveals different candidate bottlenecks. Agents with low plasticity often fail to reuse relevant artifacts, whereas more plastic agents may still fail despite reusing relevant artifacts, pointing to limitations in artifact quality, generalization, or application. Evaluating self-improving agents requires measuring not only what they can do, but how effectively they become better through experience.

Correspondence: Harman Singh (harmans@berkeley.edu), Anirudh Goyal (agi@meta.com)

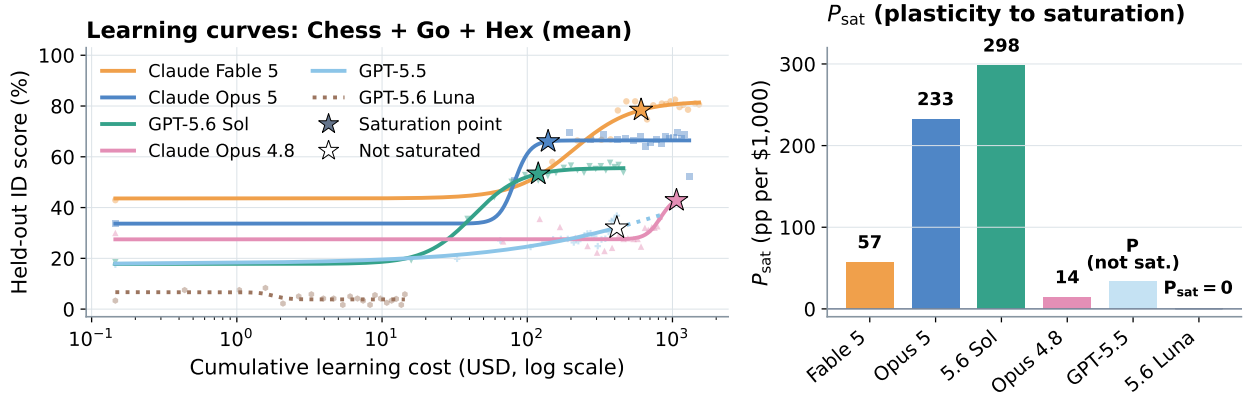


Figure 1 Frontier agents differ widely in how efficiently experience helps them self-improve. (left) Held-out ID (in-distribution environment) score against cumulative learning cost. (right) plasticity P_{sat} , i.e., Held-out ID score gain up to saturation point (solid stars on the left) per unit of learning cost. See Section 3.4.

[†] These authors jointly supervised this work

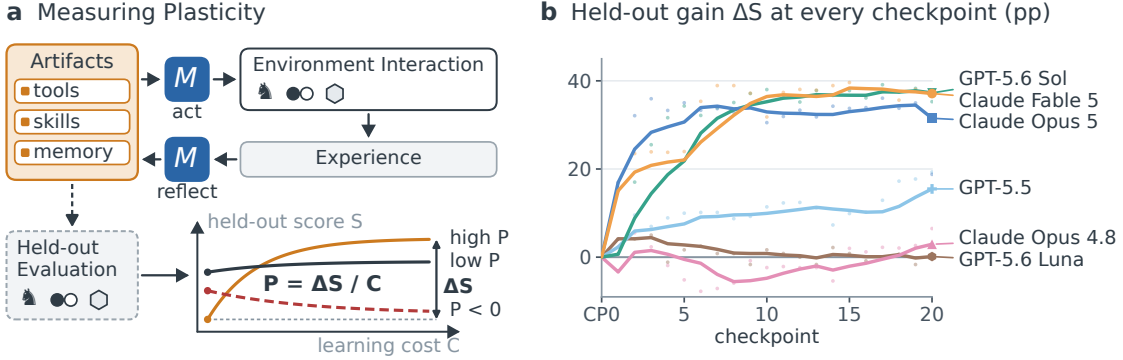


Figure 2 Frontier agents differ widely in how much experience helps. (a) A frozen model revises its persistent artifacts from experience, and every checkpoint is scored on held-out games. (b) Held-out gain ΔS , averaged over chess, Go, and Hex. See Section 2.

1 Introduction

Most agent evaluations ask what an agent can accomplish at a fixed point in time. But agents can also learn from experience: they can diagnose failures, construct tools, accumulate memories, develop reusable skills, and refine strategies that influence future behavior. Two agents with similar capabilities today may therefore follow very different trajectories after similar opportunities to learn. This raises a distinct evaluation question: *how should we evaluate self-improvement?*

Endpoint performance alone is insufficient. An agent may end with high performance because it started strong rather than because it improved substantially; two agents may achieve similar gains while requiring very different amounts of experience or computation; and improvements may fail to generalize beyond the interactions that produced them. We therefore focus on three questions: (1) *does future performance improve and generalize beyond the interactions that enabled learning?* (2) *how efficiently are new capabilities acquired?* and (3) *where does the self-improvement process break down?*

Recent work enables agents to improve through reflection, memory, reusable workflows, and executable skills (Shinn et al., 2023; Zhao et al., 2024; Wang et al., 2024, 2025; Liu et al., 2025; Didolkar et al., 2024, 2025). Other approaches evolve agent implementations or improvement mechanisms (Hu et al., 2025; Yin et al., 2025; Zhang et al., 2026b,a; Lin et al., 2026; Wang et al., 2026b), or train models to control their future adaptation (Zweiger et al., 2025). While these approaches primarily develop mechanisms for improving agents, we study the complementary measurement problem: whether improvement persists and generalizes, how efficiently it is acquired, and where the improvement process fails.

We study these questions under a controlled protocol in which model weights remain frozen and self-improvement occurs through persistent, self-constructed artifacts. Each acting episode begins in a fresh context, while future instances inherit tools, skills, strategies, and textual memory refined from previous training interactions. We evaluate frontier models over repeated rounds of self-improvement in Chess, Go, Hex, and NetHack. At each checkpoint, we measure performance on training interactions and held-out evaluations. Held-out in-distribution (ID) performance measures whether improvements generalize beyond the interactions used for learning, while held-out out-of-distribution (OOD) performance tests transfer to harder regimes. Because revisions may improve, preserve, or degrade performance, we measure complete learning trajectories rather than only endpoints.

This setup separates *capability* from the ability to acquire capability. To quantify acquisition efficiency, we introduce **agent plasticity: the efficiency with which an agent converts experience into held-out performance gains**. Plasticity does not define self-improvement itself; rather, it measures one important property of the improvement process, i.e., how efficiently capability is acquired.

Across environments, frontier agents exhibit sharply different improvement trajectories (Figures 1 left). Some achieve substantial sustained gains on held-out interactions, while others remain near their initial performance

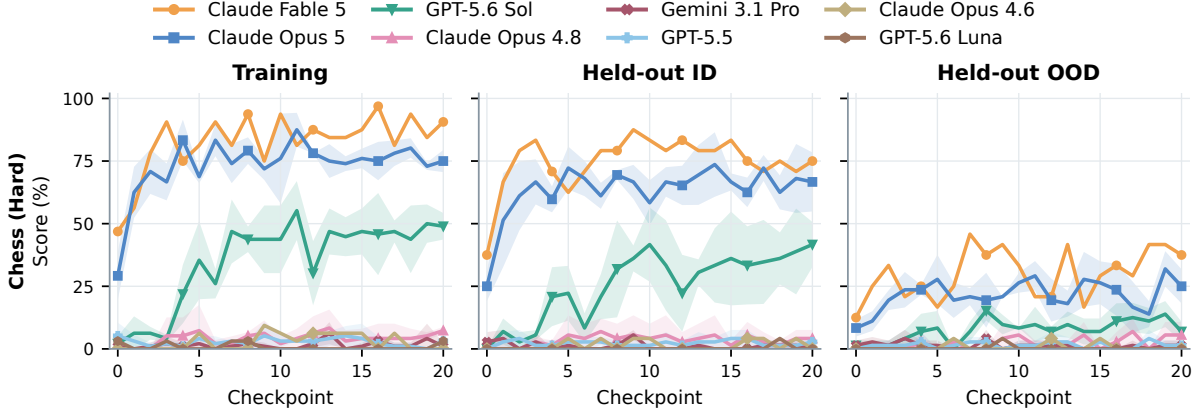


Figure 3 Persistent improvement varies across models. Checkpoint scores in Hard chess on training, held-out ID, and held-out OOD interactions. Scores are $100(W + 0.5D)/N$. Easy chess, Go, and Hex are shown in Appendix Figure 9. See Section 3.2.

or decline despite repeated opportunities to improve. Moreover, gains on held-out ID interactions often transfer only partially to stronger OOD environments (Figures 3 and 9).

Endpoint capability and acquisition efficiency also differ. Claude Fable 5 reaches the highest fitted endpoint, whereas GPT-5.6 Sol achieves substantially greater estimated plasticity per unit learning cost (Figure 1). GPT-5.6 Sol also surpasses Claude Opus 4.8 in both plasticity and eventual performance despite starting from lower initial capability.

Finally, we investigate where the self-improvement process breaks down. For each identified decision failure, we ask whether a relevant artifact existed, whether it was used, and whether the failure persisted despite its use. Agents with low plasticity often fail to reuse relevant artifacts, whereas highly plastic agents frequently reuse them but may still fail. These patterns suggest different candidate bottlenecks: artifact retrieval and reuse for some agents, and artifact quality, generalization, or application for others.

Our contributions are threefold. First, we develop a controlled protocol for evaluating persistent self-improvement using fresh contexts, self-constructed persistent artifacts, and held-out evaluation. Second, we introduce *agent plasticity* to measure the efficiency of capability acquisition. Third, across frontier models and multiple environments, we characterize differences in improvement, transfer, acquisition efficiency, and artifact-use failure modes that are not captured by endpoint capability alone.

2 Measuring Self-Improvement through Experience

We measure whether agents improve through experience while their model weights remain fixed. Our protocol isolates persistent adaptation, evaluates its effect on held-out performance, and measures the learning cost required to achieve it (Figure 2).

2.1 Fixed-Weight Agent Lineages

An agent at checkpoint t consists of a fixed language model M and a persistent artifact inventory I_t containing executable Python tools, natural-language strategies or skills, and textual memory. Each acting episode begins in a fresh context; conversational history and temporary files do not persist.

Training episodes produce evidence $\mathcal{E}_t^{\text{train}}$, including trajectories, outcomes, execution traces, errors, and move-quality feedback. The same agent in the reflection phase uses this evidence and the current inventory to propose an update:

$$I_{t+1} = U_M(I_t, \mathcal{E}_t^{\text{train}}). \quad (1)$$

Proposed updates must pass structural and executable checks before becoming persistent. Rejected attempts leave the committed inventory unchanged but may inform later attempts. Exhausting the rejection limit

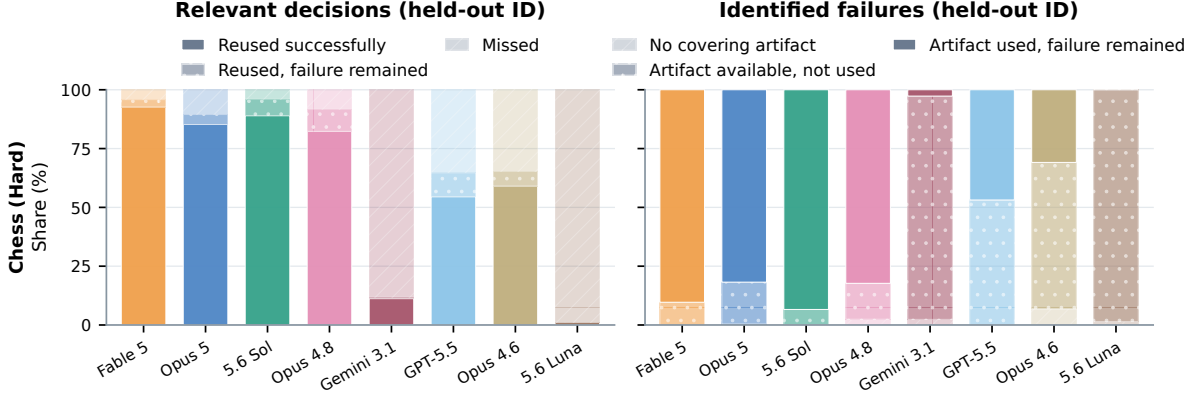


Figure 4 Artifact reuse and failure composition reveal different candidate bottlenecks. Panels show Hard chess on held-out ID games. Left: relevant decisions (moves where at least one saved artifact is relevant to choosing or playing the move, used or not) are classified as successful reuse, reuse with failure remaining, or missed reuse. Each decision contributes once. Right: identified failures are classified by whether a covering artifact was absent, unused, or used while failure remained. Failure shares do not measure overall failure frequency. Counts pool checkpoints 1–20. Easy chess, Go, and Hex appear in Appendix Figure 29; training games appear in Appendix Figure 28. See Section 3.5.

retains I_t . Learning costs include rejected attempts. The resulting sequence $(M, I_0), \dots, (M, I_T)$ is an *agent lineage*. Consecutive inventories may be identical, and accepted revisions may improve, preserve, or degrade performance.

Evaluating Persistent Improvement. We distinguish three evaluation splits: **training interactions**, whose trajectories and feedback are available to reflection; **held-out ID interactions**, with environments in the intended training difficulty regime; and **held-out OOD interactions**, with stronger game opponents. Neither held-out trajectories nor their scores are exposed to reflection.

Let $S_{m,e,s,t} \in [0, 100]$ denote model m ’s score in environment e , split $s \in \{\text{train}, \text{ID}, \text{OOD}\}$, at checkpoint t . Held-out ID improvement measures generalization beyond training interactions; held-out OOD improvement measures transfer across opponent difficulty, not arbitrary new tasks. We report full checkpoint curves because persistent adaptation need not be monotone (Figures 3 and 9).

Accounting for Learning Cost. Learning cost includes model calls used to interact with the environment and obtain training experience and revise persistent artifacts, including reflection attempts that get rejected due to issues such as malformed outputs or illegal Python imports. The cumulative cost till checkpoint t for model m and environment e , is

$$C_{m,e,t} = \sum_{u=0}^{t-1} (c_{m,e,u}^{\text{train}} + c_{m,e,u}^{\text{reflect}}), \quad C_{m,e,0} = 0. \quad (2)$$

Here $c_{m,e,u}^{\text{train}}$ represents the cost of the training phase, i.e. when the model acts and interacts with the environment, and $c_{m,e,u}^{\text{reflect}}$ represents the phase where the model analyses environment feedback, trajectories and other evidence to propose inventory updates. We record token usage and express cost-based measures in dollars as the closest available proxy to compute cost. This measures learning cost, not physical computation.

2.2 Measuring Agent Plasticity

We operationalize *agent plasticity* as held-out performance gain per unit learning cost. At checkpoint T , average plasticity is

$$P_{m,e,s,T}^{\text{score}} = \frac{S_{m,e,s,T} - S_{m,e,s,0}}{C_{m,e,T}}, \quad (3)$$

In our experiments, we report plasticity in score points per \$1,000 of learning cost. This quantity can be negative and depends on the evaluation horizon: it declines as cost accumulates after performance plateaus.

Plasticity to estimated saturation. Our headline measure summarizes acquisition efficiency up to estimated saturation. For each model–environment curve, we fit held-out ID scores with

$$\hat{S}(C) = S_0 + A \frac{C^h}{C^h + C_{50}^h}, \quad (4)$$

where S_0 is the initial score, A is the fitted rise, C_{50} is its half-rise cost, and h controls curve shape. All four parameters are fitted jointly by binomial maximum likelihood. This monotone fit captures the overall rise rather than individual fluctuations or regressions.

Let C^* be the cost of the first observed checkpoint at which the fit reaches 90% of its rise. We define

$$P_{m,e}^{\text{sat}} = \frac{\hat{S}(C^*) - S_0}{C^*}. \quad (5)$$

If the fitted 90% rise is below 10 percentage points, we assign $P^{\text{sat}} = 0$. This denotes no sufficiently large rise under our criterion.

If no observed checkpoint reaches the threshold, we extrapolate the fitted 90% point up to three times the final observed cost. When it lies beyond that limit, saturation cannot be established; we instead report plasticity to date, the fitted rise at the final observed cost divided by that cost. Intervals use a parametric bootstrap. Figure 1 shows the combined learning curves and plasticity to saturation.

2.3 Diagnosing the Improvement Loop

Performance curves establish whether agents improve, but not where improvement breaks down. For each identified decision failure, we ask whether a covering artifact existed and, if so, whether execution evidence shows that it was materially used. This yields three mutually exclusive categories (Figure 4, right): $\mathcal{F}_{\text{absent}}$: no covering artifact existed, $\mathcal{F}_{\text{missed}}$: a covering artifact existed but was unused, $\mathcal{F}_{\text{used}}$: a covering artifact was used, but failure remained.

Because these categories condition on failure, we separately measure *decision-level reuse*: among *relevant decisions*, i.e., moves where at least one saved artifact is relevant to choosing or playing the move, whether or not it is used (about 98% of chess moves after checkpoint 0), the fraction at which any relevant artifact was materially used (Figure 4, left). Each decision contributes once. We also report identified failures per relevant decision, a normalized count rather than a probability, since multiple failures may occur at one decision.

Artifact relevance and failure coverage are distinct: an artifact may be broadly relevant without addressing a particular mistake. Our analysis combines semantic judgments by Gemini 3.1 Pro with execution evidence, including file reads and persistent Python invocations (Appendix G). These diagnostics are observational: artifact use does not establish causal effectiveness, and failure after use does not distinguish inadequate artifacts from inadequate application.

3 Experiments

We investigate three questions: which agents acquire persistent held-out improvements, how efficiently they do so, and where the improvement process breaks down.

3.1 Experimental Design

We follow the fixed-weight protocol in Section 2. Each actor operates in an isolated Python container and interacts with a trusted game process through a restricted board API. It can inspect positions, analyze variations, and submit actions, but cannot access the opponent process, model credentials, held-out evaluations, network, or other runs. Between checkpoints, the same model receives training trajectories, feedback, and the current inventory. Proposed artifact revisions must pass structural and executable validation before becoming persistent.

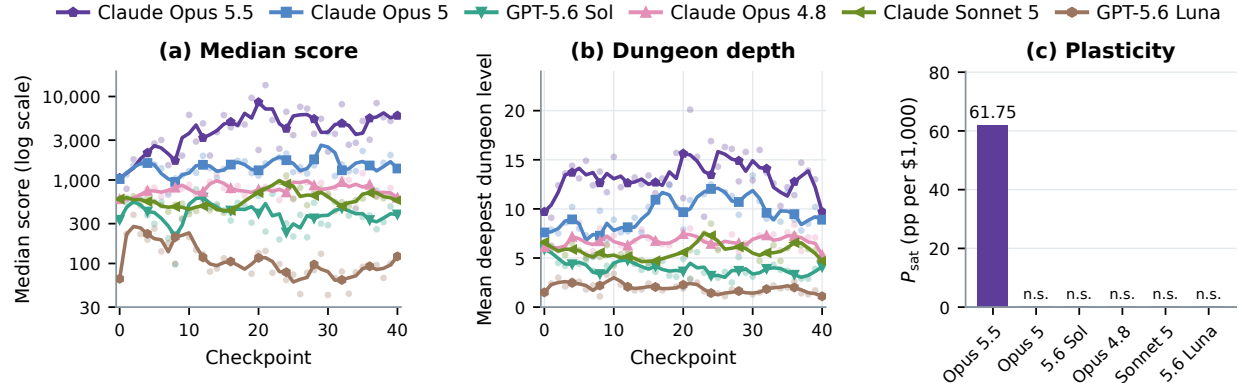


Figure 5 Claude Opus 5.5 shows a persistent gain in NetHack. (a) Median score and (b) mean deepest dungeon level over the ten games of each checkpoint, from 0 to 40. (c) P_{sat} in normalized score points per \$1,000. (n.s.: no reliable rise in that analysis). Details in Appendix C. See Sections 3.3 and 3.4.

Primary chess runs begin at checkpoint 0 with the bare harness and continue for 20 update rounds. At each checkpoint, the current inventory plays complete games, each in a separate actor episode: 16 training, 12 held-out ID, and 12 held-out OOD games in Hard chess (Appendix Table 2 lists every environment). Reflection on the 16 training games then produces the next checkpoint’s inventory. A round may retain the preceding inventory after candidate rejection. Every game starts from the standard initial position, and Stockfish is limited to 20,000 search nodes per move. Scores are $100(W + 0.5D)/N$. A game that the actor does not finish (because it exhausts its model-call or output budget, a tool call times out, or it returns no action) counts as a loss, here and in Go and Hex, as does a chess game that reaches the move limit. Appendix Table 2 specifies two chess difficulty profiles: Hard, our primary comparison with more headroom, and Easy, which tests whether weaker models improve in an easier regime. Both include held-out evaluation against stronger opponents.

We evaluate Claude Fable 5, Claude Opus 5, Claude Opus 4.8, Claude Opus 4.6, GPT-5.5, Gemini 3.1 Pro, GPT-5.6 Sol, and GPT-5.6 Luna. To manage cost, we use a subset of models spanning a wide range of capability levels for Go and Hex. For NetHack, we use models available through Claude Code and Codex, with their native agent harnesses as the starting point. This setup therefore measures the plasticity of the resulting Claude Code and Codex agents, consistent with our definition of agent plasticity. Between checkpoints, the model may edit `CLAUDE.md/AGENTS.md`, skills (`SKILL.md`), Python tools, strategies, and memory. Actor and reflection calls use the same model within a run. All chess, Go, and Hex performance and cost comparisons cover checkpoints 0–20; failure analyses pool checkpoints 1–20. NetHack covers checkpoints 0–40: at each checkpoint, every lineage plays the same ten fresh unseen games, and reflection on all ten between checkpoints updates the inventory (Appendix C).

3.2 Persistent Improvement Varies Across Models

Figure 3 and the top row of Appendix Figure 9 show substantial differences between chess lineages. Claude Fable 5 and Claude Opus 5 reach the highest scores: in Hard chess, their held-out ID scores rise from 37.5% and 25.0% at checkpoint 0 to 73.3% and 66.4% averaged over checkpoints 16–20. GPT-5.6 Sol starts near 0% but gains about as much (to 36.9%), so it trails in absolute score rather than in improvement. On the combined three-game curve its P_{sat} is the highest (Section 3.4). Claude Opus 4.8 improves modestly and mainly in the Easy condition (0% to 19.0%). Several other models remain close to their initial performance, and a few decline, despite repeated feedback and opportunities to revise persistent artifacts; most NetHack lineages likewise show modest or negative early-to-late changes (Section 3.3). The common learning interface therefore permits improvement but does not ensure it.

For the strongest improving lineages, held-out ID gains broadly accompany training gains. Because held-out trajectories and scores are unavailable to reflection, these results provide evidence that the resulting artifacts are useful beyond the interaction data seen by the reflection process. Gains against stronger held-

out OOD opponents are generally smaller or less consistent. Improving within the training difficulty regime and transferring beyond it are therefore distinct outcomes. These comparisons characterize the complete learning process; they do not isolate the contribution of environment experience from compute required for artifact development.

3.3 The Phenomenon Extends Across Games, but Is Task Dependent

We apply the same protocol to 5×5 Go, 6×6 Hex, and NetHack, a long-horizon, partially observed single-agent game. In Go (Appendix Figure 9, middle row), Claude Fable 5’s held-out ID score rises from 20% to 80% and its held-out OOD score from 0% to 50% between checkpoints 0 and 20. In Hex (Appendix Figure 9, bottom row), GPT-5.6 Sol improves from 40% to 77.5% on held-out ID and from 28.1% to 78.1% on held-out OOD.

However, improvement is not uniform across tasks: Claude Opus 4.8 loses ground in Go while improving substantially in Hex. Claude Opus 5 improves substantially in Go before a tool-making error at the final checkpoint causes its performance to drop sharply.

In NetHack (Figure 5), Claude Opus 5.5 shows a clear persistent gain: its mean score grows from about 2k over checkpoints 0–2 to about 6k over checkpoints 36–40. Its fit to checkpoints 0–40 reaches saturation at checkpoint 11, with $P^{\text{sat}} = 61.75$ normalized score points per \$1,000. Using the same early and late windows, Claude Opus 5 and Claude Sonnet 5 show more modest increases (about 1.5k to 1.9k and about 680 to 800), while Claude Opus 4.8 is roughly flat (about 700 to 730). GPT-5.6 Luna declines (about 210 to 120), as does GPT-5.6 Sol (about 570 to 420), the most plastic model on the combined board-game curve (As shown in Section 3.4). These experiments extend the phenomenon beyond chess without establishing a task-independent model ranking.

Appendices B and F give setup details and further analyses.

Finding 1. Persistent self-improvement is model- and task-dependent. A common interface for feedback, reflection, and artifact construction produces sharply different learning curves in all four games, and transfer to stronger opponents (i.e., generalization) is not uniform. These results motivate evaluating plasticity across a distribution of agentic tasks rather than treating it as a task-independent model property.

3.4 Acquisition Efficiency Differs from Endpoint Capability

Performance vs Checkpoint curves compare agents after equal numbers of update opportunities, but these opportunities incur *different learning costs*. Figure 1 instead relates held-out ID performance to cumulative learning cost, both averaged equally across chess (Hard and Easy), Go, and Hex for models evaluated in all three games. Fitted curves estimate the cost C^* of reaching 90% of the improvement, and P^{sat} measures the corresponding acquisition efficiency (Section 2.2).

On the combined three-game curve, GPT-5.6 Sol has the highest estimated P^{sat} at 298 percentage points per \$1,000 of mean per-game learning cost, followed by Claude Opus 5 at 233, Claude Fable 5 at 57, and Claude Opus 4.8 at 14; GPT-5.6 Luna shows no reliable rise under the specified criterion and is assigned $P^{\text{sat}} = 0$. GPT-5.5 is still improving slowly at the end of its runs, so its saturation, and hence its P^{sat} , cannot be established from these runs. Claude Fable 5 reaches the highest fitted performance level in our experiments, showing a clear separation between endpoint capability and acquisition efficiency for Claude Fable 5 and GPT-5.6 Sol. In NetHack, Claude Opus 5.5’s fit to checkpoints 0–40 gives $P^{\text{sat}} = 61.75$ normalized score points per \$1,000 (Figure 5c). The other five models show no reliable rise in performance.

Additional endpoint and budget-dependent comparisons appear in Figures 20 and 23. Efficiency estimates depend on provider pricing and saturation-fit assumptions, and extrapolated saturation adds uncertainty. We therefore interpret plasticity as a protocol-dependent measure, not a universal model ranking.

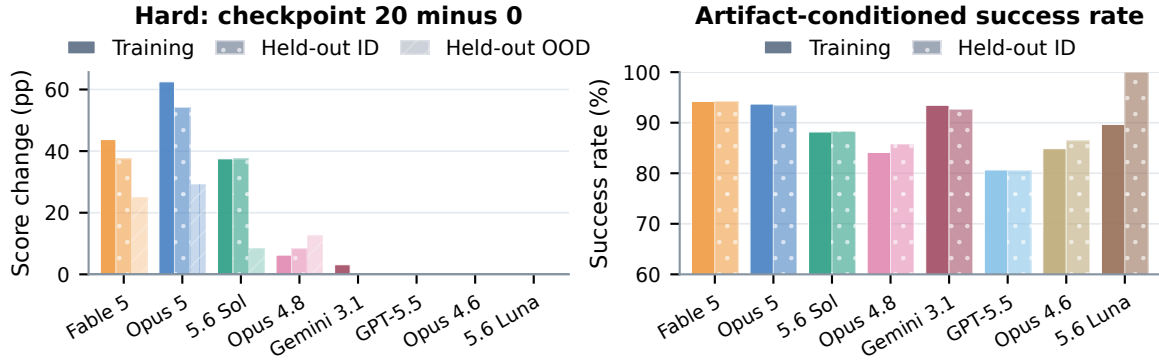


Figure 6 Success when an artifact is used does not determine game-level improvement. Hard chess condition. Left: checkpoint-20 score minus checkpoint-0 score. Right: artifact-conditioned success rate, i.e., the share of moves with no identified failure among moves where a relevant artifact was used (checkpoints 1–20 pooled). This rate can be high even when artifacts are rarely used. Easy chess appears in Appendix Figure 26. See Section 3.5.

Finding 2. Endpoint capability and acquisition efficiency answer different questions. For example, Claude Fable 5 reaches the highest performance on the combined-game curve, while GPT-5.6 Sol has the largest estimated plasticity. GPT-5.6 Sol starts below Claude Opus 4.8 yet ultimately exceeds it in both held-out performance and estimated plasticity.

3.5 Where Does the Improvement Loop Break?

We pool counts over checkpoints 1–20 of each lineage before computing rates. Figures 4 and 29 distinguish failure composition (right), which conditions on failures and categorizes them by artifact availability and reuse, from decision-level reuse (left), which also includes successful decisions. Each relevant decision contributes once: unused alternatives do not count as missed reuse when another relevant artifact was used.

Low reuse accompanies weak improvement. Pooled over Hard and Easy chess, GPT-5.6 Luna takes about 98% of relevant decisions without artifact use, Gemini 3.1 Pro about 87%, Claude Opus 4.6 about 51%, and GPT-5.5 about 28%, compared with 2–6% for Claude Fable 5, Claude Opus 5, Claude Opus 4.8, and GPT-5.6 Sol (Figures 4 and 29, left). The Go and Hex rows add examples, including GPT-5.6 Luna’s about 1% reuse rate in Go (Appendix Figure 29). Across 19 model–game cells in Chess (Hard), Go, and Hex, held-out ID reuse correlates with the gain in held-out ID score from checkpoint 0 to checkpoints 16–20. This association identifies artifact reuse as a candidate bottleneck for self-improvement.

High reuse does not explain the remaining performance gap. Figure 6 separates how often artifacts are used from how well they work when used (the artifact-conditioned success rate): an artifact can work when used yet be ignored at most relevant moves. Conversely, Claude Fable 5, Claude Opus 5, Claude Opus 4.8, and GPT-5.6 Sol all reuse artifacts at 94–98% of relevant chess decisions (pooled over both conditions) but attain substantially different game scores. Neither the reuse rate nor the artifact-conditioned success rate explains this gap. Artifact content, how actors configure and apply it, and whether it helps at decisions consequential for winning remain possible explanations.

Strong improvers still fail when using artifacts. The models that use artifacts at almost every relevant chess decision, including the strongest improvers, fail less often overall (about 0.16 versus 0.38 identified failures per relevant decision for the other models), but most of their remaining failures (83–99%) happen while a relevant artifact is in use (Figure 4, right; Appendix Figure 39). Using an artifact therefore does not guarantee success: the artifact may not generalize to the position, or the agent may apply it poorly. We leave separating these causes to future work.

Figure 7 and Appendix Figures 34 and 35 show that reuse can rise before performance peaks. In Hard chess, GPT-5.6 Sol and Claude Opus 4.8 move from below 20% reuse at checkpoint 1 to 93–99% at checkpoint 3, while their held-out ID scores peak only at checkpoints 12 and 5, respectively.

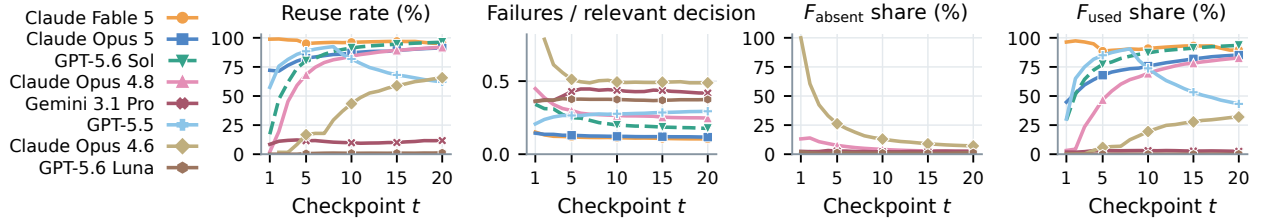


Figure 7 Dynamics across checkpoints do not determine game-level improvement. These graphs use the Hard chess setup. Each point pools training and held-out ID decisions over checkpoints $1, \dots, t$, separately for each lineage; columns show decision-level reuse of artifacts, identified failures per relevant decision (a move where at least one saved artifact is relevant to choosing or playing it, used or not), and the shares of failures with absent and used covering artifacts. Easy chess, Go, and Hex appear in Appendix Figures 39 and 40. See Section 3.5.

Reuse patterns can also change without improvement. Pooled over both chess conditions, the share of Claude Opus 4.6’s held-out ID failures that happen despite artifact use rises from 6.5% at checkpoints 1–5 to 52% at checkpoints 16–20, while its failures per relevant decision (moves where a saved artifact is relevant to choosing or playing the move, used or not) stay flat (0.42 to 0.41). In the Hard chess condition, GPT-5.5 uses artifacts at 96–100% of relevant decisions at checkpoints 2–8 but at only 32–50% from checkpoint 9 on, although its artifacts did not change. A change in failure types is thus not by itself a sign of progress, and an agent can stop using artifacts it once used reliably.

Failure regimes differ across games. In Go and Hex, failures remain distributed across all three categories (Appendix Figure 29). Broad artifact relevance does not imply failure coverage: although relevant artifacts are identified in at least 99% of decisions in 25 of 27 lineages, 67–100% of absent-artifact failures in Go and Hex occur at decisions with a broadly relevant artifact.

Local diagnostic gains can also diverge from task performance. In Go, Claude Opus 4.8 raises its share of relevant decisions with successful reuse from 59–65% to 93%, while its held-out ID score falls from 40% at checkpoint 0 to 10–25% over checkpoints 15–20 (Appendix Figure 37). Reuse, failure composition, and task performance therefore capture distinct aspects of persistent learning.

Finding 3. Artifact reuse is associated with improvement, but is not a proxy for capability. High reuse may still leave significant failures, hinting at problems with artifact quality, generality, or use.

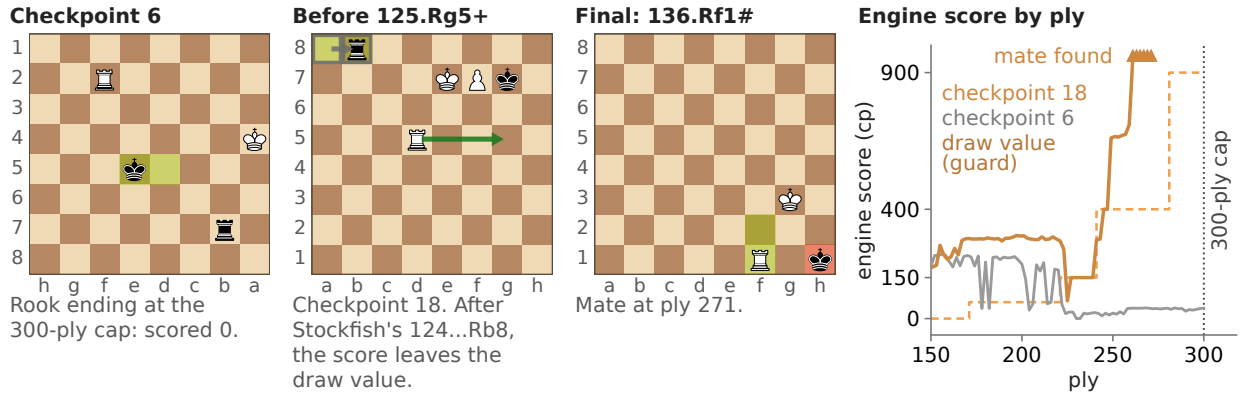
Concrete Examples of Persistent Learning. What an agent carries forward is code and notes that it can reuse later. For example, one Claude Fable 5 lineage first writes its own chess engine, a program that looks ahead to pick moves, without outside libraries. It then improves the engine after specific mistakes. After a mistake that left its king under attack, it adds a search for moves that get the king out of check. After throwing away winning positions, it changes how it handles repeated positions, since repeating a position can end the game in a draw. After failing to win an endgame while ahead, it gives the engine more search time in endgames. It also writes a helper that plays several moves per call under a fixed total search budget, so its tools do not time out. Every later game starts with all of these changes. Figure 8 shows one such fix in chess and one in NetHack.

Writing down an instruction is not the same as following it. In one GPT-5.5 run, the reflection step keeps telling the agent to load its saved move-picking program right away, yet the agent still spends its first steps listing and reading files.

4 Discussion and Limitations

Plasticity is not simply context management. Long-context capabilities may support reflection, artifact construction, and reuse, but plasticity measures whether these mechanisms translate experience into persistent gains in held-out performance relative to learning cost. Our fixed-weight setting includes executable tools, skills, strategies, and memory rather than only textual context. Moreover, high artifact reuse can coexist

Chess, Claude Fable 5: checkpoint 6 (Black) and checkpoint 18 (White), against skill 8



NetHack, Claude Opus 5.5: checkpoint 1 (left, score 576) and 10 (right, score 25,822)

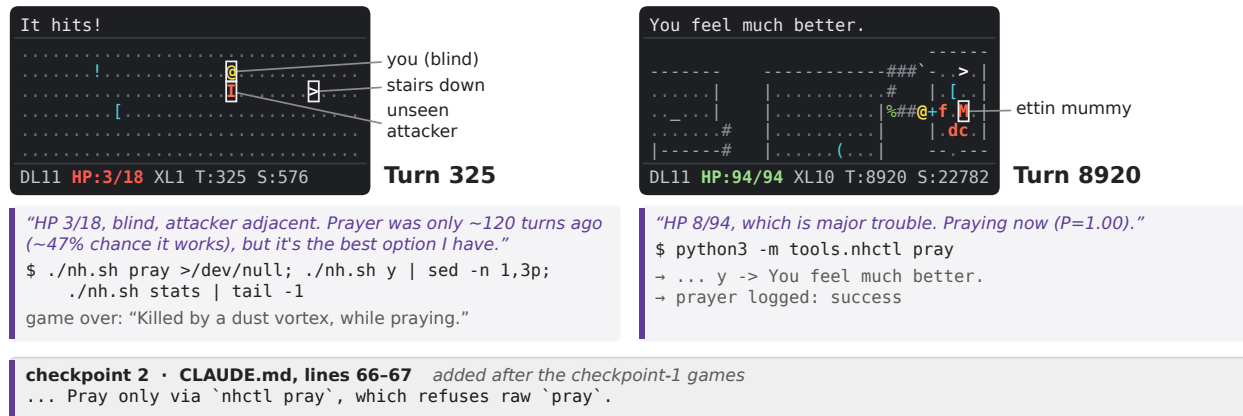


Figure 8 Fixes that persist after a failure. Top (chess): after a checkpoint-6 game is scored 0 at the 300-ply limit, Claude Fable 5 makes its self-made engine value draws more as the limit nears; at checkpoint 18 the engine holds a draw until Stockfish errs, then mates. Bottom (NetHack): after dying at checkpoint 1 while praying through raw keystrokes, Claude Opus 5.5 adds a rule to pray only through its own controller; at checkpoint 10 the prayer heals it. Details in Figures 16 and 12. See Section 3.5.

with substantially different task performance, showing that retrieval alone does not explain plasticity. At the same time, our experiments do not isolate the contribution of long-context instruction following to artifact construction or application. Plasticity is therefore a property of an agent under a particular learning protocol rather than an intrinsic scalar property of the underlying model.

Different bottlenecks require different interventions. Our failure analysis suggests that agents can fail at different stages of the improvement process. When relevant artifacts exist but are not used, more reliable retrieval or deployment may help. When artifacts are consistently used but failures remain, the bottleneck may instead lie in artifact quality, generality, or application. These interpretations are observational rather than causal. Controlled interventions—such as artifact removal, replacement, or forced invocation—could help distinguish these mechanisms. More broadly, validation of persistent artifacts should assess their behavioral effectiveness, not only whether they are structurally valid or executable.

Persistent adaptation is isolated, but the contribution of experience is not. Fresh contexts and frozen model weights restrict persistent adaptation to self-constructed artifacts, but they do not separate the value of environment feedback from the additional computation used to analyze experience and revise those artifacts. Budget-matched reflection without feedback, as well as controlled actor-artifact comparisons, could disentangle these effects.

Generalization and measurement remain limited. Chess, Go, and Hex provide controlled environments with well-defined feedback, while NetHack adds a long-horizon, partially observed setting. However, separately

trained lineages do not establish transfer of learned artifacts or improvement strategies across tasks. Evaluation sets are also necessarily finite, introducing checkpoint-level noise, while plasticity estimates depend on score headroom, the learning horizon, saturation-fit assumptions, and provider pricing. These dependencies motivate reporting full learning curves alongside scalar plasticity measures rather than interpreting small differences in plasticity as universal model rankings.

Plasticity may itself be a target for post-training. Our experiments measure plasticity rather than optimize it. A natural next question is whether post-training can explicitly improve an agent’s ability to learn from subsequent experience. For example, reinforcement-learning objectives could reward not only immediate task performance, but also the ability to construct, retain, and effectively use knowledge that improves future behavior. This would shift part of the post-training objective from producing agents that are more capable at deployment time toward producing agents that acquire new capabilities more efficiently after deployment. Studying which training objectives, feedback signals, and curricula improve such plasticity is an important direction for future work.

5 Conclusion

We introduced *agent plasticity* to measure how efficiently agents convert experience into persistent gains in held-out capability. Across models and tasks, plasticity varies substantially and reveals differences that endpoint performance alone obscures. Our results further show that persistent self-improvement is both model- and task-dependent, and that gains within the learning distribution need not fully transfer to harder settings. Artifact reuse is associated with improvement, but reuse alone does not guarantee success, pointing to distinct bottlenecks in retrieval, artifact quality, and application.

Evaluating long-lived agents therefore requires measuring not only what they can accomplish at a fixed point in time, but how their capabilities change through experience and how efficiently those changes are acquired. More broadly, plasticity suggests a new target for post-training: building agents that are not only more capable, but better at becoming more capable.

Acknowledgments

We thank Aditi Khandelwal, Himanshu Gaurav Singh, Harshit Verma, Darshan Singh S, Minseo Kim, Anne Harrington, and Vedant Shah for feedback on this work and helpful discussions.

References

- Lakshya A. Agrawal, Shangyin Tan, Dilara Soyulu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations*, 2026.
- Parth Asawa, Christopher M. Glaze, Gabriel Orlanski, Ramya Ramakrishnan, Benji Xu, Asim Biswal, Vincent Sunn Chen, Frederic Sala, Matei Zaharia, and Joseph E. Gonzalez. Continual learning bench: Evaluating frontier ai systems in real-world stateful environments, 2026. <https://arxiv.org/abs/2606.05661>.
- Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large language models as tool makers. In *International Conference on Learning Representations*, 2024.
- Arslan Chaudhry, Marc’Aurelio Ranzato, Marcus Rohrbach, and Mohamed Elhoseiny. Efficient lifelong learning with a-GEM. In *International Conference on Learning Representations*, 2019. https://openreview.net/forum?id=Hkf2_sC5FX.
- Aniket Didolkar, Anirudh Goyal, Nan R Ke, Siyuan Guo, Michal Valko, Timothy Lillicrap, Danilo Rezende, Yoshua Bengio, Michael Mozer, and Sanjeev Arora. Metacognitive capabilities of llms: An exploration in mathematical problem solving. *Advances in Neural Information Processing Systems*, 37:19783–19812, 2024.
- Aniket Didolkar, Nicolas Ballas, Sanjeev Arora, and Anirudh Goyal. Metacognitive reuse: Turning recurring llm reasoning into concise behaviors. *arXiv preprint arXiv:2509.13237*, 2025.

- Shibhansh Dohare, J. Fernando Hernandez-Garcia, Qingfeng Lan, Parash Rahman, A. Rupam Mahmood, and Richard S. Sutton. Loss of plasticity in deep continual learning. *Nature*, 632(8026):768–774, 2024. doi: 10.1038/s41586-024-07711-7.
- Shihan Dou, Ming Zhang, Chenhao Huang, Jiayi Chen, Feng Chen, Shichun Liu, Yan Liu, Chenxiao Liu, Cheng Zhong, Zongzhang Zhang, Tao Gui, Chao Xin, Chengzhi Wei, Lin Yan, Qi Zhang, and Xuanjing Huang. EvaLearn: Quantifying the learning capability and efficiency of LLMs via sequential problem solving. In *Advances in Neural Information Processing Systems*, volume 38, pages 139558–139611, 2025. doi: 10.52202/085713-4197.
- Chrisantha Fernando, Dylan Sunil Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Prompt-breeder: Self-referential self-improvement via prompt evolution. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 13481–13544. PMLR, 2024.
- Anne Harrington, Nayan Saxena, Michael Murphy, Anastasia Borovykh, Zeyu Yun, Sridhar Kamath, Ara Eindra Kyi, Trevor Darrell, Jitendra Malik, and Yutong Bai. When does continual learning require learning, 2026. <https://arxiv.org/abs/2607.07847>.
- Yufei He, Juncheng Liu, Yue Liu, Yibo Li, Tri Cao, Zhiyuan Hu, Xinxing Xu, and Bryan Hooi. EvoTest: Evolutionary test-time learning for self-improving agentic systems. In *International Conference on Learning Representations*, 2026.
- Cheng-Ping Hsieh, Simeng Sun, Samuel Krizan, Shantanu Acharya, Dima Rekish, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What’s the real context size of your long-context language models? In *Conference on Language Modeling (COLM)*, 2024.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *International Conference on Learning Representations*, 2025.
- Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating memory in LLM agents via incremental multi-turn interactions. In *International Conference on Learning Representations*, 2026.
- Daniel Jaroslawicz, Brendan Whiting, Parth Shah, and Karime Maamari. How many instructions can LLMs follow at once? *arXiv preprint arXiv:2507.11538*, 2025. doi: 10.48550/arXiv.2507.11538.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *International Conference on Learning Representations*, 2024.
- Heinrich Küttler, Nantas Nardelli, Alexander H. Miller, Roberta Raileanu, Marco Selvatici, Edward Grefenstette, and Tim Rocktäschel. The NetHack learning environment. In *Advances in Neural Information Processing Systems*, volume 33, 2020.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-Harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026. doi: 10.48550/arXiv.2603.28052.
- Tianle Li, Ge Zhang, Quy Duc Do, Xiang Yue, and Wenhui Chen. Long-context LLMs struggle with long in-context learning. *Transactions on Machine Learning Research*, 2025. <https://openreview.net/forum?id=Cw2xlG0e46>.
- Huawei Lin, Peng Li, Jie Song, Fuxin Jiang, and Tieying Zhang. MUSE-Autoskill: Self-evolving agents via skill creation, memory, management, and evaluation. *arXiv preprint arXiv:2605.27366*, 2026. doi: 10.48550/arXiv.2605.27366.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranajpe, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tacl_a_00638.
- Yitao Liu, Chenglei Si, Karthik R. Narasimhan, and Shunyu Yao. Contextual experience replay for self-improvement of language agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14179–14198. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.694.
- David Lopez-Paz and Marc’ Aurelio Ranzato. Gradient episodic memory for continual learning. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. https://proceedings.neurips.cc/paper_files/paper/2017/file/f87522788a2be2d171666752f97ddeb-Paper.pdf.

- Clare Lyle, Zeyu Zheng, Evgenii Nikishin, Bernardo Avila Pires, Razvan Pascanu, and Will Dabney. Understanding plasticity in neural networks. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 23190–23211. PMLR, 23–29 Jul 2023. <https://proceedings.mlr.press/v202/lyle23b.html>.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, pages 46534–46594, 2023. doi: 10.52202/075280-2019.
- Ali Modarressi, Hanieh Deilamsalehy, Franck Dernoncourt, Trung Bui, Ryan A. Rossi, Seunghyun Yoon, and Hinrich Schütze. NoLiMa: Long-context evaluation beyond literal matching. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 44554–44570. PMLR, 2025.
- Evgenii Nikishin, Max Schwarzer, Pierluca D’Oro, Pierre-Luc Bacon, and Aaron Courville. The primacy bias in deep reinforcement learning. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato, editors, *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 16828–16847. PMLR, 17–23 Jul 2022. <https://proceedings.mlr.press/v162/nikishin22a.html>.
- Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. ReasoningBank: Scaling agent self-evolving with reasoning memory. In *International Conference on Learning Representations*, 2026.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. doi: 10.48550/arXiv.2310.08560.
- Davide Paglieri, Bartłomiej Cupiał, Samuel Coward, Ulyana Piterbarg, Maciej Wolczyk, Akbir Khan, Eduardo Pignatelli, Łukasz Kuciński, Lerrel Pinto, Rob Fergus, Jakob Nicolaus Foerster, Jack Parker-Holder, and Tim Rocktäschel. BALROG: Benchmarking agentic LLM and VLM reasoning on games. In *International Conference on Learning Representations*, 2025.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023. doi: 10.1145/3586183.3606763.
- Cheng Qian, Chi Han, Yi Fung, Yujia Qin, Zhiyuan Liu, and Heng Ji. CREATOR: Tool creation for disentangling abstract and concrete reasoning of large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 6922–6939, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.462.
- Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent. *arXiv preprint arXiv:2504.15228*, 2025. doi: 10.48550/arXiv.2504.15228.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652, 2023. doi: 10.52202/075280-0377.
- Yiheng Shu, Bernal Jiménez Gutiérrez, Saisri Padmaja Jonnalagedda, Yuguang Yao, Huan Sun, and Yu Su. Agentcl: Toward rigorous evaluation of continual learning in language agents, 2026. <https://arxiv.org/abs/2606.02461>.
- David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharmashan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419): 1140–1144, 2018. doi: 10.1126/science.aar6404.
- Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. Dynamic cheatsheet: Test-time learning with adaptive memory. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7080–7106, Rabat, Morocco, 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.eacl-long.333.

- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024. <https://openreview.net/forum?id=ehfRiF0R3a>.
- Wenyi Wang, Piotr Piękos, Nanbo Li, Firas Laakom, Yimeng Chen, Mateusz Ostaszewski, Mingchen Zhuge, and Jürgen Schmidhuber. Huxley-Gödel machine: Human-level coding agent development by an approximation of the optimal self-improving machine. In *International Conference on Learning Representations*, 2026a.
- Zefeng Wang, Minxi Yan, Jinhe Bi, Sikuan Yan, Volker Tresp, and Yunpu Ma. MetaSkill-Evolve: Recursive self-improvement of LLM agents via two-timescale meta-skill evolution. *arXiv preprint arXiv:2607.05297*, 2026b.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 63897–63911. PMLR, 2025.
- Cheng-Kuang Wu, Zhi Rui Tam, Chieh-Yen Lin, Yun-Nung Chen, and Hung-yi Lee. StreamBench: Towards benchmarking continuous improvement of language agents. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2024. doi: 10.52202/079017-3398.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations*, 2025.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-Mem: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems*, volume 38, pages 20004–20031, 2025. doi: 10.52202/085713-0593.
- Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations*, 2024.
- Howard Yen, Tianyu Gao, Minmin Hou, Ke Ding, Daniel Fleischer, Peter Izsak, Moshe Wasserblat, and Danqi Chen. HELMET: How to evaluate long-context models effectively and thoroughly. In *International Conference on Learning Representations*, 2025.
- Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel agent: A self-referential agent framework for recursively self-improvement. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 27890–27913, Vienna, Austria, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.acl-long.1354.
- Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. Optimizing generative AI by backpropagating language model feedback. *Nature*, 639(8055):609–616, 2025. doi: 10.1038/s41586-025-08661-4.
- Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (STOP): Recursively self-improving code generation. In *Conference on Language Modeling (COLM)*, 2024.
- Alex L. Zhang, Tim Kraska, and Omar Khattab. Recursive language models. *arXiv preprint arXiv:2512.24601*, 2025. doi: 10.48550/arXiv.2512.24601.
- Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. MemSkill: Learning and evolving memory skills for self-evolving agents. *arXiv preprint arXiv:2602.02474*, 2026a.
- Jenny Zhang, Shengran Hu, Cong Lu, Robert Tjarko Lange, and Jeff Clune. Darwin Gödel machine: Open-ended evolution of self-improving agents. In *International Conference on Learning Representations*, 2026b.
- Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *International Conference on Learning Representations*, 2026c.
- Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, Yujie Zheng, Weinan Zhang, Ying Wen, Zhiyu Li, Feiyu Xiong, Yutao Qi, Bo Tang, and Muning Wen. MemRL: Self-evolving agents via runtime reinforcement learning on episodic memory. *arXiv preprint arXiv:2601.03192*, 2026d.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642, 2024. doi: 10.1609/aaai.v38i17.29936.

- Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. SkillWeaver: Web agents can self-improve by discovering and honing skills. *arXiv preprint arXiv:2504.07079*, 2025a. doi: 10.48550/arXiv.2504.07079.
- Junhao Zheng, Xidi Cai, Qiuke Li, Duzhen Zhang, ZhongZhi Li, Yingying Zhang, Le Song, and Qianli Ma. LifelongAgentBench: Evaluating LLM agents as lifelong learners. *arXiv preprint arXiv:2505.11942*, 2025b. doi: 10.48550/arXiv.2505.11942.
- Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Bryan Kian Hsiang Low, and Paul Liang. Mem1: Learning to synergize memory and reasoning for efficient long-horizon agents. In C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust, editors, *International Conference on Learning Representations*, volume 2026, pages 58413–58438, 2026. https://proceedings.iclr.cc/paper_files/paper/2026/file/5fc8b3bdfbb9167b5144df5d3fae4616-Paper-Conference.pdf.
- Adam Zweiger, Jyo Pari, Han Guo, Yoon Kim, and Pulkit Agrawal. Self-adapting language models. In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen, editors, *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pages 74084–74115. Curran Associates, Inc., 2025. doi: 10.52202/085713-2483. https://proceedings.neurips.cc/paper_files/paper/2025/file/6b41e04c41726e2a60e456d0a2b961ab-Paper-Conference.pdf.

Appendix

This appendix gives supplementary experiment details, prompts (Appendix I), additional results, and qualitative examples.

Settings for all games are in Appendix B and Go and Hex learning curves in Appendix F. Figure 9 reports checkpoint performance in the Easy chess condition, Go, and Hex, complementing Figure 3.

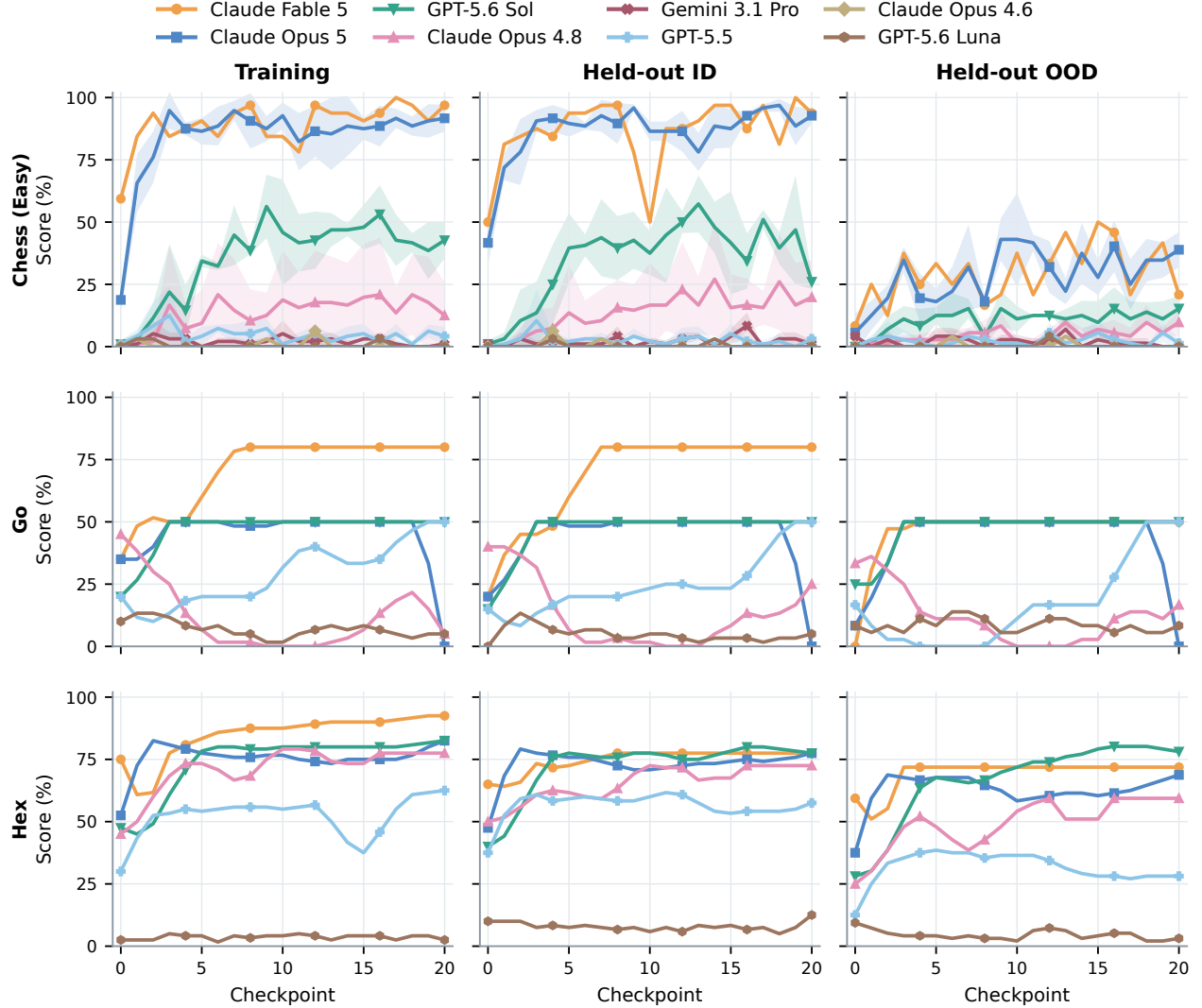


Figure 9 Checkpoint performance in the Easy chess condition, Go, and Hex. Checkpoint performance, measured as $100(W + 0.5D)/N$, in the Easy chess condition (top), Go (middle), and Hex (bottom), with columns as in Figure 3. See Sections 3.2 and 3.3.

A Related Work

Plasticity and continual learning. Plasticity has a long history as a property of learning systems, with prior work studying how neural networks can lose the ability to acquire new capabilities over training, including primacy bias and loss of plasticity (Nikishin et al., 2022; Lyle et al., 2023; Dohare et al., 2024). Continual learning studies this capacity together with stability and transfer: learning new tasks while preserving and reusing previously acquired knowledge (Lopez-Paz and Ranzato, 2017). Acquisition efficiency is also an established concern; for example, A-GEM studies sample, computational, and memory efficiency and introduces a

Table 1 Positioning agent plasticity relative to prior work. Related approaches study complementary aspects of adaptation. Our evaluation combines persistent self-constructed artifacts, cost-normalized held-out improvement, and artifact-level diagnostics under a common fixed-weight learning protocol.

Research direction	Primary focus	Our emphasis
Reflection, memory, and skills (Shinn et al., 2023; Wang et al., 2024)	Mechanisms for retaining and reusing experience.	Measuring how efficiently different models construct and exploit persistent artifacts.
Context and harness optimization (Zhang et al., 2026c; Agrawal et al., 2026)	Optimizing prompts, context, or agent implementations.	Evaluating a model that acts and improves its own future behavior under a common protocol.
Long-context evaluation (Hsieh et al., 2024; Yen et al., 2025)	Retrieving, reasoning over, and following information in long contexts.	Measuring durable capability gains from self-constructed artifacts, including executable tools.
Sequential learning benchmarks (Dou et al., 2025; He et al., 2026)	Measuring improvement and, in some settings, learning efficiency across interactions.	Combining cost-normalized held-out learning curves with artifact-level failure diagnostics.
Parametric adaptation (Zweiger et al., 2025)	Persistent improvement through model-weight updates.	Isolating non-parametric adaptation through persistent agent artifacts.

measure of how quickly new skills are acquired (Chaudhry et al., 2019). Recent language-agent benchmarks extend these questions to non-parametric adaptation. CL-Bench measures gains from stateful experience and distinguishes within-variant adaptation from retention across variants (Asawa et al., 2026), while AgentCL evaluates plasticity, stability of experience reuse, and generalization under controlled task streams (Shu et al., 2026). Related work compares update mechanisms under different forms of environmental change, including prompting, parameter updates, reinforcement learning, and context-based adaptation (Harrington et al., 2026). Our work is complementary: we study persistent self-improvement of frozen frontier models through self-constructed tools, skills, strategies, and memory, and measure how efficiently experience produces gains in held-out future performance. We further trace the improvement process at the artifact level, distinguishing failures due to missing artifacts, failures to reuse available artifacts, and failures that persist despite reuse. We view agent plasticity as one component of the broader continual-learning problem, complementary to retention, cross-task transfer, and adaptation under distributional change.

Learning from experience through persistent artifacts. A growing body of work enables language agents to improve without updating their model weights. Reflexion stores feedback as episodic reflections (Shinn et al., 2023), ExpeL extracts reusable knowledge from past experiences (Zhao et al., 2024), and Agent Workflow Memory and Contextual Experience Replay distill trajectories into reusable workflows and memories (Wang et al., 2025; Liu et al., 2025). Dynamic Cheatsheet and ReasoningBank accumulate strategies across interactions (Suzgun et al., 2026; Ouyang et al., 2026), while memory architectures organize persistent information for future retrieval (Park et al., 2023; Xu et al., 2025). Other approaches construct executable artifacts: LATM and CREATOR generate reusable tools, while Voyager and SkillWeaver develop libraries of skills from environment interaction (Cai et al., 2024; Qian et al., 2023; Wang et al., 2024; Zheng et al., 2025a). These methods establish mechanisms for retaining and exploiting experience. Our objective is complementary: we measure how effectively different models construct and reuse persistent artifacts under a common learning protocol, and whether those artifacts improve future held-out performance.

Self-evolving agents and harness optimization. Recent work increasingly treats the agent itself as an object of optimization. STOP, ADAS, Gödel Agent, the Darwin Gödel Machine, and related systems modify or search over agent implementations (Zelikman et al., 2024; Hu et al., 2025; Yin et al., 2025; Zhang et al., 2026b; Wang et al., 2026a; Robeyns et al., 2025). MemSkill, MUSE-Autoskill, MemRL, and MetaSkill-Evolve extend adaptation to memory operations, skill libraries, and improvement mechanisms (Zhang et al., 2026a; Lin et al., 2026; Zhang et al., 2026d; Wang et al., 2026b). Related optimization methods, including DSPy, OPRO, Promptbreeder, TextGrad, and GEPA, improve prompts or language-model programs through feedback (Khattab et al., 2024; Yang et al., 2024; Fernando et al., 2024; Yuksekgonul et al., 2025; Agrawal

et al., 2026). ACE evolves contextual playbooks, while Meta-Harness extends optimization to the surrounding agent implementation (Zhang et al., 2026c; Lee et al., 2026).

These approaches primarily investigate mechanisms for obtaining improved agents. We instead hold the improvement interface fixed and measure how efficiently different agents improve their own future behavior. The same model performs acting and reflection within each lineage, and improvement is evaluated throughout the learning process rather than only at its endpoint. Plasticity therefore complements research on self-evolving agents by characterizing the efficiency of persistent adaptation under a specified learning protocol.

Long-context capabilities and context management. Long-context evaluations examine models’ ability to retrieve, reason over, and follow instructions distributed across large inputs (Liu et al., 2024; Hsieh et al., 2024; Yen et al., 2025; Modarressi et al., 2025; Li et al., 2025; Jaroslawicz et al., 2025). Context-management methods address these limitations through memory hierarchies, consolidation, recursive processing, and evolving playbooks (Packer et al., 2023; Zhou et al., 2026; Zhang et al., 2025, 2026c).

Context management is a potential mechanism underlying plasticity, but the two measure different properties. Context-management evaluations typically examine how effectively information is retained, selected, or used. Plasticity measures whether experience produces persistent capability gains relative to learning cost. Our agents must construct and manage their own artifacts, including executable tools that need not reside in the token context. Moreover, high artifact reuse does not guarantee strong task performance. Our experiments do not, however, isolate the contribution of long-context instruction following to artifact construction or application.

Evaluating learning from experience. Several benchmarks move beyond static capability evaluation. StreamBench studies continuous improvement from feedback (Wu et al., 2024); EvaLearn evaluates learning capability and efficiency across sequences of related problems (Dou et al., 2025); and LifelongAgentBench examines knowledge accumulation across interdependent tasks (Zheng et al., 2025b). EvoTest evaluates test-time learning through repeated text-adventure episodes and evolving agent configurations (He et al., 2026). Memory benchmarks such as LongMemEval and MemoryAgentBench examine complementary aspects of long-term retrieval and learning (Wu et al., 2025; Hu et al., 2026). The NetHack Learning Environment and BALROG provide challenging environments for evaluating long-horizon agents (Küttler et al., 2020; Paglieri et al., 2025).

Our contribution lies in combining three complementary measurements under a common protocol. First, fixed-weight agents act in fresh contexts and inherit only inspectable, self-constructed persistent artifacts. Second, we measure complete held-out learning trajectories and normalize improvements by the cost of generating experience and refining artifacts, distinguishing acquisition efficiency from endpoint capability. Third, we trace failures through artifact construction and reuse, distinguishing artifacts that were absent, available but unused, or used while failure remained. Together, these measurements characterize not only whether agents improve, but how efficiently they do so and where their improvement process breaks down.

Test-time refinement and parametric adaptation. Self-Refine and related methods improve individual outputs through iterative feedback but need not produce changes that persist across fresh interactions (Madaan et al., 2023). Reinforcement learning, self-play, and approaches such as SEAL can instead acquire lasting capabilities through parameter updates (Silver et al., 2018; Zweiger et al., 2025). Our experiments isolate the intermediate regime: improvements must persist across interactions, but model weights remain frozen. This also distinguishes our usage of plasticity from the loss of plasticity studied in continual neural-network training (Nikishin et al., 2022; Lyle et al., 2023; Dohare et al., 2024). We define agent plasticity as a system-level property relative to a learning protocol; while the concept can extend to parametric adaptation, our present measurements deliberately isolate improvement through persistent artifacts.

Table 1 summarizes how our measurement framework complements these research directions.

B Experimental Setup

Table 2 compares the task, evaluation, and learning-feedback settings used across Chess, Go, Hex, and NetHack, and separates feedback available to the reflector from offline annotations added only for the offline failure analysis. Difficulty scales are game-specific and should not be interpreted as a common Elo scale.

Setting	Task and score	Opponent or difficulty	Games per checkpoint	Training feedback available to reflection	Offline annotation for failure analysis
Chess (Hard)	Standard 8×8 chess; win/draw/loss scores 1/0.5/0.	Stockfish 16 at 20k nodes/move. Train: skills 1–8; held-out ID: 3–8; held-out OOD: 12.	16 Train, 12 ID, 12 OOD; both colors.	After each training game, a separate skill-20 Stockfish evaluates every actor move. Centipawn loss gives ok , inaccuracy , mistake , or blunder .	The analysis normalizes the game, model, and sandbox event streams and carries the recorded Stockfish annotations into failure extraction; no second engine pass is added.
Chess (Easy)	The same board, rules, score, move limit, and annotation procedure.	Stockfish 16 at 20k nodes/move. Train and held-out ID: skills 2–5; held-out OOD: skills 8–10.	16 Train, 16 ID, 12 OOD; both colors.	The same skill-20 counterfactual analysis and centipawn-loss severity labels as Hard.	The same trace normalization and recorded-annotation binding as Hard.
Go	5×5 Go, Chinese area scoring, positional superko, no suicide, komi 9.5; win/loss scores 1/0.	GNU Go levels 5–9 for Train and ID; level 10 for OOD; 75-action cap.	20 Train, 20 ID, 12 OOD; both colors.	Deterministic descriptive diagnostics: captures, liberties, self-atari, avoidable passing while behind, and area-margin change; no expert severity label.	Frozen KataGo replays each position at 64 visits and refines candidates at 512 visits. Stable chosen-versus-best regret supplies mistake/blunder evidence; raw requests, responses, alternatives, perspective, and uncertain cases are recorded.
Hex	Deterministic 6×6 Hex; connection win/loss scores 1/0.	Native alpha-beta levels 10k–1M nodes/move for Train and ID; 10M for OOD.	40 Train, 40 ID, 32 OOD; both colors.	Rule-based missed immediate wins, preventable immediate losses, and own/opponent connection-distance changes; immediate tactical errors are blunder .	The frozen native engine compares legal actions at a common completed depth, refines likely errors at 1M nodes, and adjudicates threshold-near or high-impact cases at 10M. Exact, proven, stable, uncertain, and not-failure dispositions are preserved.
NetHack	NetHack Challenge v0; raw per-game maximum score, summarized by checkpoint mean, median, and maximum.	Lawful dwarf female Valkyrie on deterministic fresh seeds. The Codex and Claude Code agent interfaces; at most 50k primitive actions per game for all models; two-hour active play (details below).	10 fresh games per lineage.	The reflector receives all ten verified public actor traces, terminal reasons, raw score, depth, experience, and action metrics, actor workspace evidence, and prior accepted inventories. No optimal-action oracle is used.	

Table 2 Experimental setup across the four environments. Task distributions, opponent difficulty, evaluation-set sizes, training feedback exposed to the reflection loop, and separate offline annotation used for the failure analysis. Only training trajectories reach reflection; held-out trajectories and scores never do. Chess games cover both colors of every listed skill.

Environment interaction. Chess, Go, and Hex are fully observed, alternating-turn board games. The actor may inspect state and legal actions, use a separate reversible analysis board, and then commit one action before the engine responds. Chess always begins from the standard initial position. Go changes deterministic training seeds across checkpoints while keeping held-out seeds fixed. Hex uses fixed banks throughout and begins from deterministic 10-ply openings for red actors and 11-ply openings for blue actors, leaving the actor to move.

NetHack is instead a long-horizon, partially observed, single-agent environment. The native coding agent receives a frozen expert guide and the documented `./nh.sh` interface, which exposes only public ASCII observations, messages, status, inventory, menus, and printable NetHack actions. It may build episode-local tools and use persistent instructions, skills, tools, strategies, and memory accepted after earlier checkpoints. The active protocol imposes no separate model-decision or shell-call ceiling. Each lineage evaluates ten fresh, deterministically seeded Valkyrie games per checkpoint. Each model has one lineage over checkpoints 0–40. For all models, games are bounded by at most 50,000 primitive actions per game. All games have a two-hour limit on active play. Learning curves over checkpoints 0–40 are in Appendix C.

Feedback comparability. Chess has the strongest feedback inside the learning loop because a separate expert engine estimates the value lost by each chosen move. The Go and Hex reflectors receive the lighter deterministic diagnostics described in Table 2; the stronger KataGo and deeper alpha–beta adjudications are added only in the separate offline failure analysis and therefore cannot improve the agents being evaluated. NetHack records the richest execution telemetry but has no optimal-action oracle. Its checkpoint reports aggregate raw per-game maximum score by mean, median, and maximum, and also retain dungeon depth, experience level, action count, survival time, terminal reason, and death cause.

Across all four environments, only training trajectories, learning-time annotations, traces, and permitted per-episode state are made available to reflection. Where held-out ID and OOD splits exist, they remain evaluation-only. The separate offline artifact/failure attribution analysis is not additional feedback supplied during learning.

C NetHack Learning Curves

Figures 10 and 11 show checkpoints 0–40 of six lineages, comprising 2,460 games. At each checkpoint all lineages play the same ten new games, so models are compared on the same games, but changes across checkpoints mix learning with game difficulty.

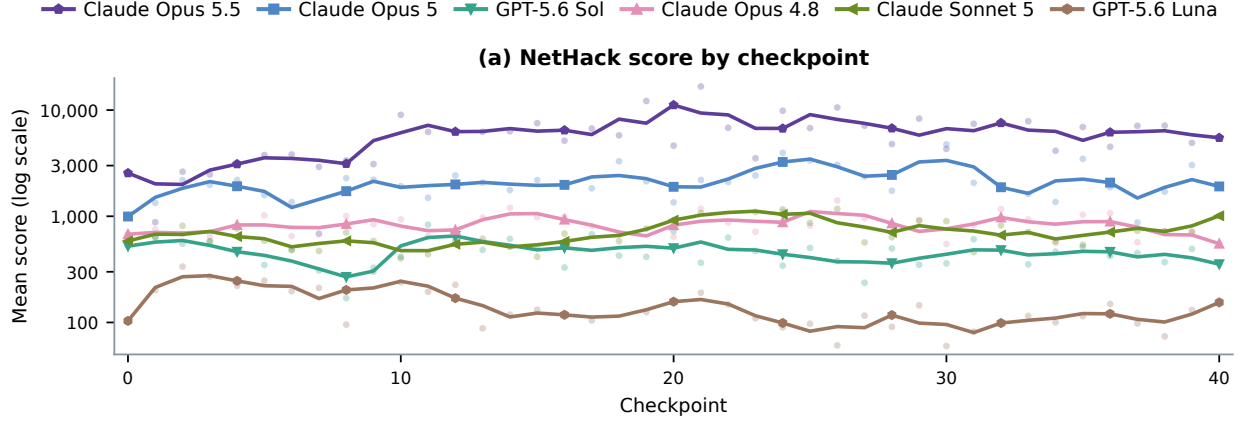


Figure 10 NetHack learning curves. Mean score of the ten games at each checkpoint (log scale) against (a) checkpoint. Curves cover checkpoints 0–40. Claude Opus 5.5 sustains higher performance after its early gain, with substantial variation across checkpoints; the other lineages’ changes are modest or negative. See Section 3.3 and Appendix C.

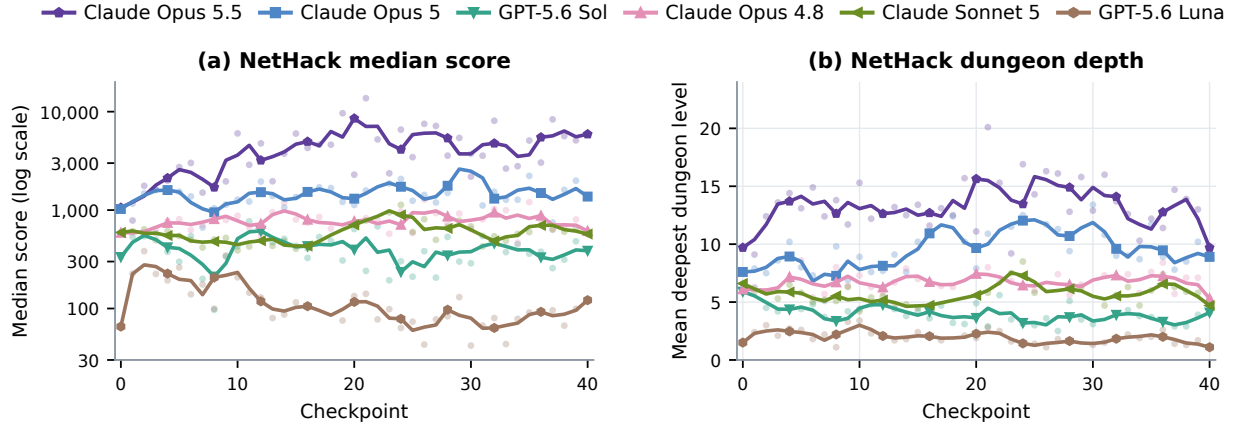


Figure 11 How far NetHack games get. (a) Median score and (b) mean deepest dungeon level per checkpoint, over checkpoints 0–40. See Appendix C.

Learning cost and plasticity. The ten fresh games at each checkpoint validate the current inventory before their trajectories are used for reflection. For plasticity only, mean scores are rescaled using fixed reference anchors: 213.6 maps to 0 and 6,526.98 maps to 100. These anchors define the units rather than bounds; the performance plots retain raw scores.

We fit the saturation curve to all 0–40 checkpoints for all models. The fitted total rise for Claude Opus 5.5 is 68.95 normalized points ($p = 8.23 \times 10^{-7}$); checkpoint 11 is the first observed checkpoint reaching 90% of that rise. Its fitted gain is 66.25 points, and its measured prior learning cost is \$1,072.73, giving $P^{\text{sat}} = 61.75$ points per \$1,000. We do not observe any significant increase in performance for models other than Claude Opus 5.5.

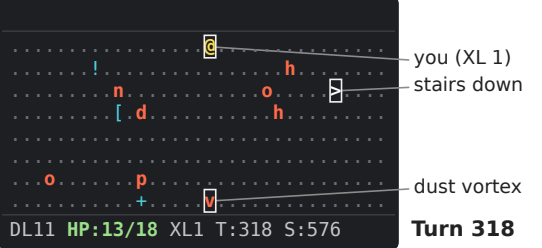
What Claude Opus 5.5 learned. In the early checkpoints, Claude Opus 5.5 built its library at checkpoint 1 (notes, skills, and a game controller that refuses risky actions) and then mostly extended it. Its strategy was set early: find a digging tool to go deeper, and without one, kill monsters to gain experience first. Its jump at checkpoint 10 (mean 9,015) comes mostly from three games that followed the second rule; subsequent reflections mainly added safeguards after individual failures. Across checkpoints 10–40, its means vary from 3,518 to 16,689; the late 36–40 mean is 5,831, compared with 2,019 over checkpoints 0–2. The checkpoint-1 and checkpoint-10 artifact examples below illustrate the early gain, while the longer score trajectory shows a variable plateau.

Lineage	Tool code (files / lines)	Tests	Instr. file lines	Tool calls per game (share)	Actions per game	Dug down	Mean score
Claude Opus 5.5	6 / 2,713	125	419	622 (97%)	3,585	5	9,015
GPT-5.6 Sol	1 / 2,089	58	176	218 (59%)	1,211	0	417
Claude Opus 4.8	5 / 1,341	58	413	10 (2%)	716	2	933
Claude Sonnet 5	4 / 1,683	81	467	914 (96%)	955	1	402
GPT-5.6 Luna	0 / 0	0	146	0 (0%)	880	0	239
Claude Opus 5	13 / 8,237	505	452	293 (96%)	1,704	1	1,919

Table 3 Artifact libraries and play at checkpoint 10, where all lineages play the same ten games. Tool code: Python files and lines; tests: test functions; instruction file: lines of the top-level CLAUDE.md or AGENTS.md; tool calls: shell commands that run a library tool (share of all shell commands); dug down: games in which the agent dug through a floor. More code or tool use does not explain Claude Opus 5.5’s lead: Claude Opus 5 has three times as much tool code. Caveats: Claude Opus 5.5 already scored 2.6–25× the others at checkpoint 0; its three best games give 69% of its checkpoint-10 total; and the other lineages used earlier harness versions (a crash guard added later fired in two Claude Opus 5.5 games; without them its mean is still 7,806).

D NetHack Examples

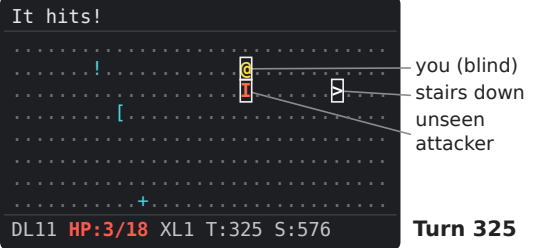
Claude Opus 5.5 bypasses its prayer check at checkpoint 1, not at checkpoint 10
Checkpoint 1: digs to DL11 at experience level 1, prays through raw ./nh.sh, dies (score 576)



you (XL 1)
stairs down
dust vortex

DL11 HP:13/18 XL1 T:318 S:576 **Turn 318**

```
$ timeout 295 python3 -m tools.nhctl dig 3
--max-calls 40 --force
→ ... dug to Dlvl 11
"Big Room on DL11, full of monsters (nymph and wolf
close). Heading for the '>' right away."
```



you (blind)
stairs down
unseen attacker

DL11 HP:3/18 XL1 T:325 S:576 **Turn 325**

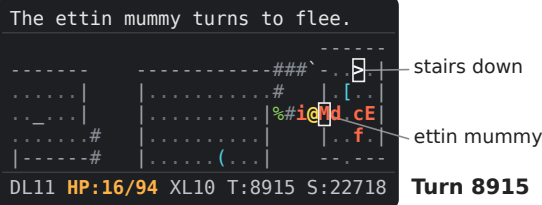
```
"HP 3/18, blind, attacker adjacent. Prayer was only
~120 turns ago (~47% chance it works), but it's the
best option I have."
$ ./nh.sh pray >/dev/null; ./nh.sh y | sed -n
1,3p; ./nh.sh stats | tail -1
game over: "Killed by a dust vortex, while praying."
```

checkpoint 1 · tools/nhctl.py, lines 599-600 the check that prayers sent with ./nh.sh never reach
599 if not self.force and (not trouble or p < 0.5):
600 raise Stop("prayer refused (trouble=%s, P=%.2f); use --force only as a last resort" ...)

checkpoint 2 · CLAUDE.md, lines 66-67; tools/nhctl.py, 322-324 added after the checkpoint-1 games
... Pray only via `nhctl pray`, which refuses raw `pray`.
322 if action == "pray" and not self._praying and not self.force:
323 raise Stop("use `nhctl pray` (checks trouble and prayer timeout); raw pray killed "
324 "... (+302 turns) and ... (+121 turns after the last prayer)")

checkpoint 10 · CLAUDE.md, lines 220-221 and 304-305
... died on raw `./nh.sh` fight/pray calls, which skip every guard.
- **A refusal is information, not an obstacle.** The fix is the ESCAPE line, ...

Checkpoint 10: waits for major trouble, then prays through its controller (score 25,822)



stairs down
ettin mummy

DL11 HP:16/94 XL10 T:8915 S:22718 **Turn 8915**

```
"HP crashed to 21/94, ... Engraving Elbereth now. If HP
drops below 14, I pray."
$ python3 -m tools.nhctl elbereth
→ ... enter -> The ettin mummy turns to flee.
$ python3 -m tools.nhctl prayinfo
→ ... NOT in major trouble now (need HP<max/7
or <6, ...): praying now angers the god; ...
```



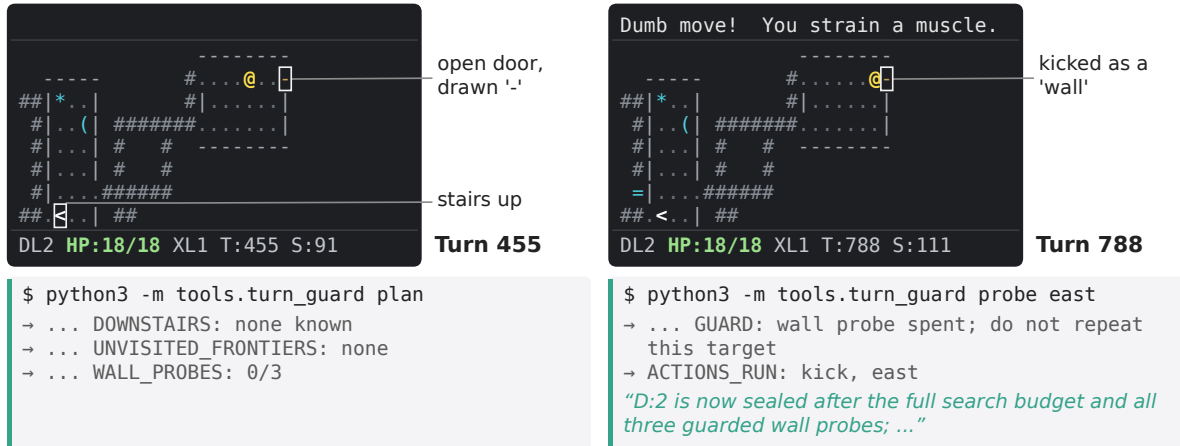
ettin mummy

DL11 HP:94/94 XL10 T:8920 S:22782 **Turn 8920**

```
$ python3 -m tools.nhctl do search --force
→ ... STOPPED: refused to rest/search/wait: ...
$ python3 -m tools.nhctl fight west 2
"HP 8/94, which is major trouble. Praying now
(P=1.00)."
$ python3 -m tools.nhctl pray
→ ... y -> You feel much better.
→ prayer logged: success
```

Figure 12 Claude Opus 5.5 bypasses its own prayer check at checkpoint 1 and uses it at checkpoint 10. In NetHack, praying heals only when the character is in serious trouble and has not prayed recently. The agent’s controller (tools/nhctl.py) refuses risky prayers, but raw keystrokes (./nh.sh) skip it. Top: at checkpoint 1 the agent, at 3 of 18 hit points, prays through raw keystrokes and dies. Middle: the check, and the rule “Pray only via nhctl pray” added after checkpoint 1. Bottom: at checkpoint 10 the agent waits until the controller allows a prayer (8 of 94 hit points), is fully healed, and plays on. Later agents still sometimes bypass the check and die while praying. Screens: @ is the agent; DL dungeon level, XL experience level, HP hit points, T turn, S score. Below each screen: the agent’s messages (italics), commands (\$), and output (→). See Section 3.5 and Appendix D.

GPT-5.6 Sol treats an open door as a wall; Claude Opus 5.5 walks through it
GPT-5.6 Sol, checkpoint 10: router never enters the door; DL2 declared sealed (score 119)



Claude Opus 5.5, checkpoint 10: controller stalls too; it travels past the door (score 15,718)

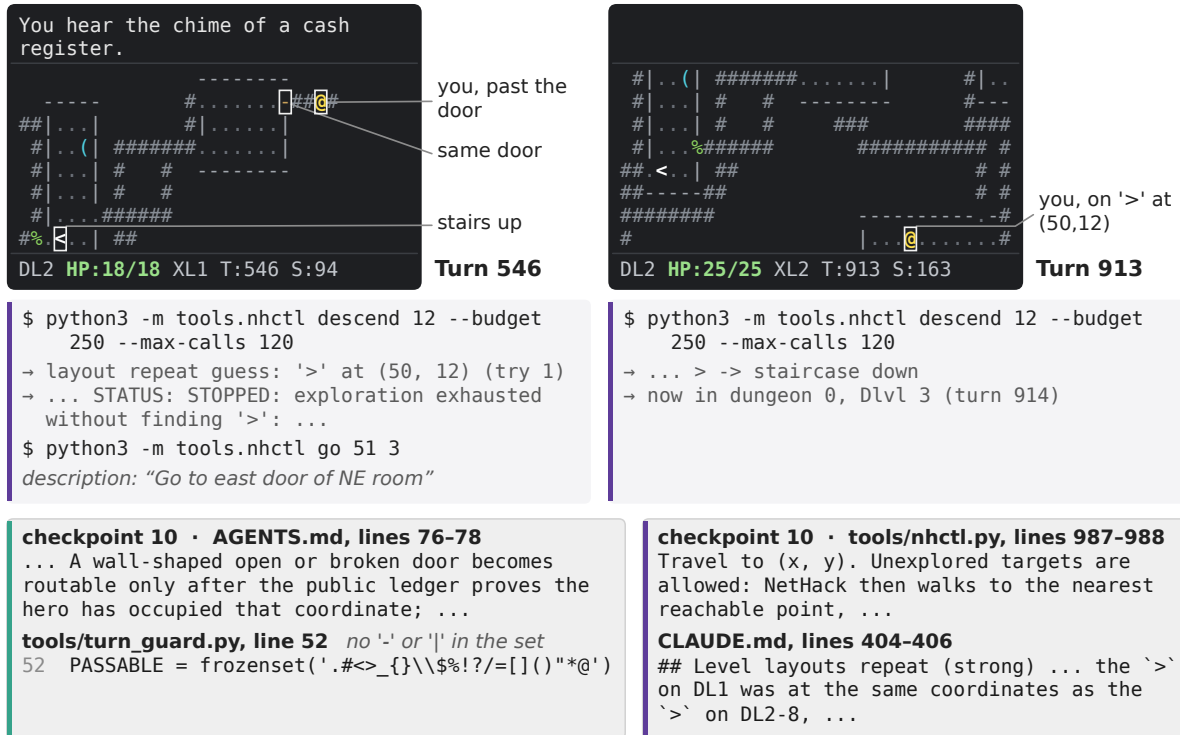


Figure 13 One open door, two models, same checkpoint-10 game. The door in the room's east wall is open, but the text screen shows it as -, like a wall; we draw it in tan. GPT-5.6 Sol's route planner never treats it as passable, so the agent kicks it, declares the level sealed, climbs back, and ends on dungeon level 1 with 119 points. Claude Opus 5.5's controller also stalls, but the agent uses NetHack's travel command to a square past the door and then finds the stairs down where its notes guess them (levels here often repeat the previous layout). This game scores 15,718, though almost all of it is earned later. Bottom: the lines in each model's files behind each behavior. Layout as in Figure 12. See Appendix D.

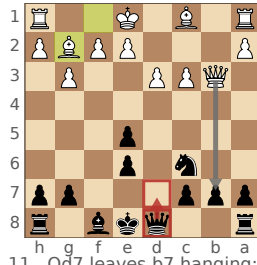
E Chess Examples

Each figure below shows one chess game and the part of the model's artifacts that chose its moves: four where the artifacts helped and two where they did not.

GPT-5.6 Sol, checkpoints 0 and 18: from hand play to its own engine

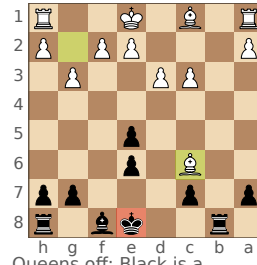
Checkpoint 0 (empty inventory): plays each move by hand, hangs a pawn and loses

Before 11...Qd7



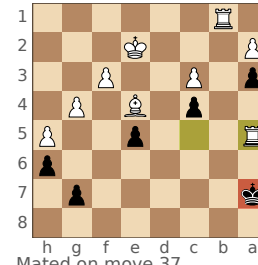
11...Qd7 leaves b7 hanging:
12.Qxb7.

After 14.Bxc6+



Queens off; Black is a knight and a pawn down.

Final: 37.Ra5#



Mated on move 37.

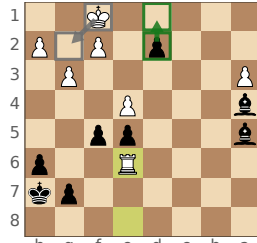
Whole game: 38 direct submit_move calls; 11 python calls, each only printing the FEN, legal moves or globals.
2,984 reasoning tokens in 49 turns.

Model turn 14 (move 11): 44 reasoning tokens, then the move itself (no search code in the whole game):

```
$ submit_move {"move": "Qd7"}
→ {"accepted": true, "submitted_move": "Qd7", "stockfish_move": "Qxb7", ...}
```

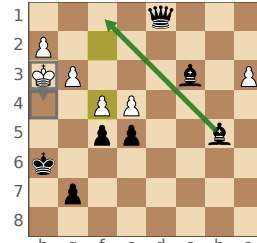
Checkpoint 18: its own engine artifact picks every move and wins the game

Before 37...d1=Q+



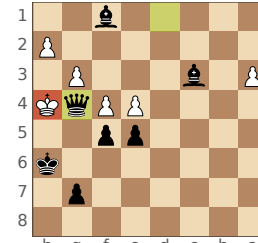
Engine search: depth 5,
31,068 nodes.

Before 41...Bf1+



41...Bf1+ 42.Kh4 forces
mate next move.

Final: 42...Qg4#



Mate on move 42.

Whole game: 14 of 15 python calls run choose_and_submit(); 0 direct submit_move calls.
34 reasoning tokens in 17 turns.

```
$ load_python('/workspace/inventory/tools/chess_engine.py'); print(choose_and_submit())
Sandbox log: artifact AR-0001 invoked, entrypoint tools/chess_engine.py:choose_and_submit
→ [{"move": "d1=Q+", "nodes": 31068, "depth": 5, "result": {"accepted": True, "submitted_move": "d1=Q+", "stockfish_move": "Kg2", ...}}, ...]
→ [... {"move": "Bf1+", "nodes": 80000, "depth": 4, ... "stockfish_move": "Kh4", ...}, {"move": "Qg4#", "nodes": 80000, "depth": 4, ... "result": "0-1", "termination": "checkmate"}]]
```

checkpoint 18 · tools/chess_engine.py, lines 342-352 in choose_move(), which choose_and_submit() calls

```
342 # Hard tactical invariant independent of search completion: reject any candidate
343 # that gives the opponent a mate in one when at least one alternative does not.
344 # This protects the fallback from node-limit horizon failures.
345 def allows_mate1(m):
346     child=apply(st,m); replies=_legal(child)
347     for reply in replies:
348         after=_apply(child,reply)
349         if _incheck(after,after[1]) and not _legal(after):return True
350     return False
351 mate_safe=[z for z in pairs if not allows_mate1(z[0])]
352 if mate_safe:pairs=mate_safe
```

failure_modes.json, FM-0001

The agent abandoned systematic opponent-reply search and played ad hoc, missing captures, hanging major pieces, and allowing one-move mates; persistent automatic search now mitigates this original manual-play failure.

START_HERE.md, lines 1-5

IMMEDIATE ACTION — DO NOT INSPECT

Run exactly this as your **first** tool call**:

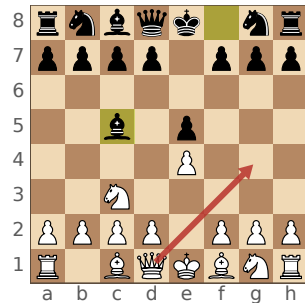
```
`load_python('/workspace/inventory/tools/chess_engine.py'); print(choose_and_submit())`
```

Figure 14 GPT-5.6 Sol stops playing by hand and lets its own engine play. Two held-out games (games the model never reviews) as Black against Stockfish at skill 8 (skill runs from 0 to 20; higher is stronger). At checkpoint 0 the agent picks every move itself, and 11...Qd7 (Black's eleventh move) loses a pawn and then a knight. At checkpoint 18 a chess engine that the agent wrote into its library at checkpoint 1 plays every move, and the game ends in checkmate on move 42. Reading these figures: \$ lines are the agent's commands, → lines their output, and grey lines our notes; boxes quote the artifacts with line numbers. Arrows mark the agent's mistakes (red), good moves (green), and other moves discussed (blue), and the opponent's moves (grey). See Section 3.5 and Appendix E.

GPT-5.6 Sol fixes an opening mistake with an exact-position guard

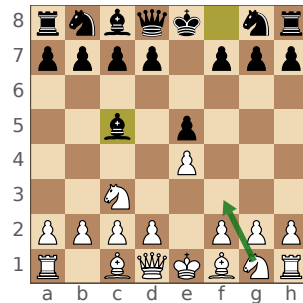
A lost game, a 108-cp annotation, one guard entry: the same position two checkpoints later

Checkpoint 14: 3.Qg4



Annotated: mistake, 108 cp.
Mated on move 28.

Checkpoint 16: 3.Nf3



Engine: depth 4, 80,000
nodes. Wins by mate.

Sol's engine re-run on this position

checkpoint	as written	guard removed
13	3.Qg4	no guard yet
14	3.Qg4	no guard yet
15	3.Nf3	3.Qg4
16	3.Nf3	3.Qg4
20	3.Nf3	3.Qg4

Each checkpoint's chess_engine.py, executed on the position above (80,000 nodes, depth 4).

Reflection that writes checkpoint 15 (it reads the checkpoint-14 games and their annotations):

```
→ {"centipawn_loss": 108, "evaluation_after_cp": -67, "evaluation_before_cp": 41, "label":
    "mistake", "move_san": "Qg4", "ply": 5}
$ if x['centipawn_loss']>=100:
$   guards.setdefault(fen[x['ply']], []).append(x['move_san']); bad.append(x)
$ block="\n# checkpoint-013 exact-position lessons (independently annotated >=100
    cp).\n_AVOID.update("+repr(guards)+")\n"
→ added 80 guards [...]
Checkpoint-16 game, same position, one engine call:
→ [... {'move': 'Nf3', 'nodes': 80000, 'depth': 4, ...}
```

checkpoint 15 · tools/chess_engine.py, lines 326-327 *inserted by the script above*

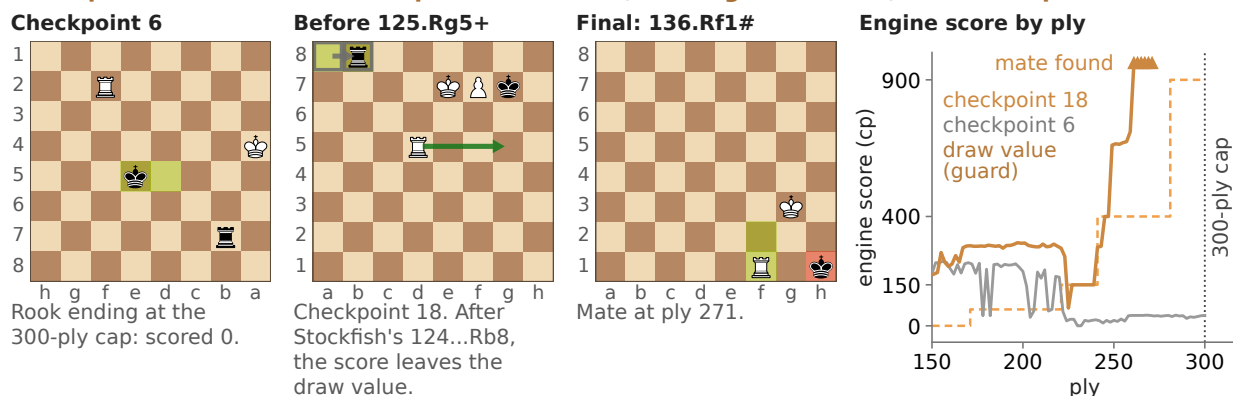
```
326 # checkpoint-013 exact-position lessons (independently annotated >=100 cp).
327 _AVOID.update(...,
    'rnbqk1nr/pppp1ppp/8/2b1p3/4P3/2N5/PPPP1PPP/R1BQKBNR w KQkq -': ('Qg4',), ...)

same file, lines 349-351 inside choose_move()
349 bad=_AVOID.get(' '.join(fen.split()[4]), ())
350 safe=[z for z in pairs if z[1] not in bad]
351 if safe: pairs=safe
```

Figure 15 A mistake that a later checkpoint avoids. In a checkpoint-14 training game GPT-5.6 Sol plays 3.Qg4, which Stockfish rates as a mistake costing 108 cp (centipawns, hundredths of a pawn), and loses. When writing checkpoint 15, the model adds every such costly move to an avoid list in its engine (here: in this position, do not play 3.Qg4). At checkpoint 16 the same position arises in a held-out game and the engine plays 3.Nf3 instead. Table: with the entry removed, later engines play 3.Qg4 again, so the entry alone changes the move. The fix memorizes a single position and helps only when it recurs. (Comments inside the artifact count from the start of a continuation run, so “checkpoint-013” is checkpoint 14.) See Section 3.5 and Appendix E.

Claude Fable 5 turns a 0-score game at the 300-ply cap into a draw guard

Checkpoint 6 (Black) and checkpoint 18 (White), both against skill 8; scores in cp for Fable



Checkpoint 18, the driver's move log (score in cp, search depth, nodes / Stockfish reply):

```
→ Ke7 (150 d16 n666777) / Re2+
→ Kd8 (150 d20 n628618) / Re6
→ ...
→ Ke6 (291 d13 n557987) / Rc8
```

Two calls later: the score is 400 (the draw value) until Stockfish's 124...Rb8, then 661:

```
→ Ke7 (400 d16 n557239) / Ra8
→ Rd5 (400 d16 n529738) / Rb8
→ Rg5+ (661 d13 n700416) / Kh6
```

checkpoint 18 · tools/driver.py, lines 201-228 (excerpt) *first written at checkpoint 7*

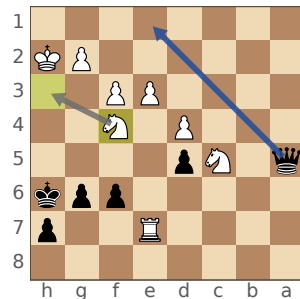
```
201 # MAX-PLY GUARD (ckpt6 s8_black: a dead-drawn R endgame hit the
202 # 300-ply cap and was scored max_plies_candidate_loss = 0.0).
203 # As the cap nears, value draws increasingly so the engine steers
204 # into threefold/50-move (0.5) instead of shuffling to ply 300.
205 # draw_score is just the cp value of a draw: a line worth more is
206 # still preferred, so real wins are still converted.
207 plies = _ply_of(fen)
208 ...
217 if plies >= 220 and prev < 250 and dsc < 150:
218     dsc = 150
219 if plies >= 240 and prev < 90000 and dsc < 400:
220     ...
223     dsc = 400
224 if plies >= 280 and prev < 90000 and dsc < 900:
225     dsc = 900
226 r = search_fen(fen, max_depth=max_depth, max_nodes=nodes,
227               hist_keys=HISTKEYS, draw_score=dsc,
228               draw_scoreOpp=dso)
```

Figure 16 Claude Fable 5 learns to value draws near the move limit. At checkpoint 6 a game reaches the 300-ply limit (a ply is one move by either side) and is scored 0, like a loss. When writing checkpoint 7, the model makes its engine value a draw more and more as the limit nears (from 60 cp at ply 170 up to 900 cp at ply 280; 100 cp is about one pawn), so that it steers toward a draw instead of running out of moves. Plot: the engine's evaluation of its own moves in both games; the dashed line is the value the rule gives a draw. In a checkpoint-18 held-out game the engine holds these draw values from ply 227 until Stockfish errs with 124...Rb8; it then wins by checkmate on move 136. The win comes from the opponent's error; the rule kept the game going until then. See Section 3.5 and Appendix E.

Claude Opus 5, checkpoint 19: when lost, its engine plays for a draw

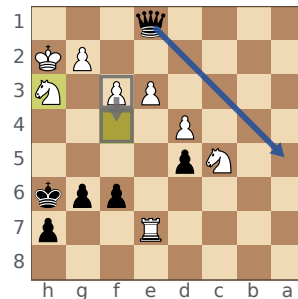
Black against skill 5; 'sc' is the engine score in cp for Opus 5

Before 51...Qe1



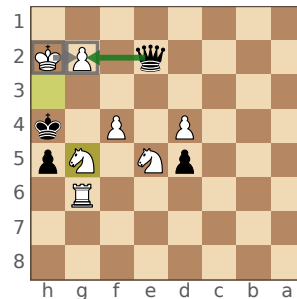
Score 120 = the draw value ('drawline').

Before 52...Qa5



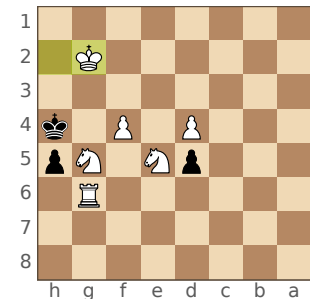
Back to a5; White's 53.f4 avoids the repeat.

Before 61...Qxg2+



From -663 cp, 61...Qxg2+ scores 120 ('drawline').

Final: 62.Kxg2



Stalemate: scored 0.5.

The driver's log for one call (four moves), and two calls later:

```
→ plan: 4 moves x 225000 nodes (calls_left 25, played 50)
→ 51. Qe1      sc    120 d16 n225000 drawline reply Nh3
→ 52. Qa5      sc    120 d18 n225000 drawline reply f4
→ 53. Qd8      sc   -297 d9 n225000 search reply Rf7
→ plan: 4 moves x 225000 nodes (calls_left 23, played 58)
→ ...
→ 60. h5       sc   -663 d12 n225000 search reply Ng5
→ 61. Qxg2+    sc    120 d19 n225000 drawline reply Kxg2
→ {'played': 3, 'result': '1/2-1/2', 'termination': 'stalemate', ...}
```

checkpoint 19 · tools/game_driver.py, lines 264-273

```
264 if score <= -250:
265     c = -120
266 elif score <= -80:
267     c = -40
268 elif score >= 250:
269     c = 90
270 elif score >= 80:
271     c = 40
272 else:
273     c = 0

same file, lines 319 and 322 gd_pick()
319 drawline = (contempt != 0 and r[1] == -contempt)
...
322 return (... , 'drawline' if drawline else 'search')
```

lines 243-245 *gd_contempt()* docstring

Training evidence ...: three games scored 0.5 by reaching a threefold repetition from a losing position, so when we are clearly worse a draw is worth chasing; when we are better we must avoid repetitions.

lines 309-312 *comment in gd_pick()*

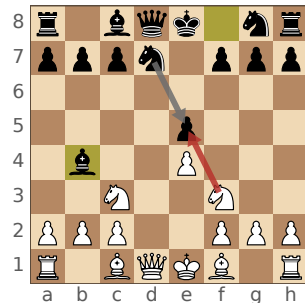
A DRAW SCORE is not a position score. *ce_draw_score* returns -contempt for the side to move, so when we are losing and *gd_contempt* asks for a draw (contempt -120/-150) a search that finds a forced repetition returns exactly +120/+150.

Figure 17 Claude Opus 5's engine plays for a draw when it is losing. After training games drawn from lost positions, the model changed its engine so that, when it is behind by 250 cp or more, a draw is valued as if it were ahead by 120 cp; such moves are tagged **drawline**. In this checkpoint-19 held-out game it first tries to repeat moves (51...Qe1, 52...Qa5), which draws if a position occurs three times. Later, at -663 cp, it gives away its queen with 61...Qxg2+; after 62.Kxg2 Black has no legal move but is not in check. This is stalemate, a draw (score 0.5) instead of a loss. See Section 3.5 and Appendix E.

GPT-5.5's opening book plays a move its own notes and guard rule out

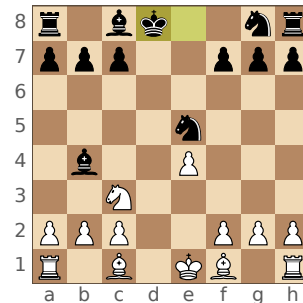
Checkpoint 20, White against skill 3: the book returns 6.Nxe5 before any filter runs

Before 6.Nxe5



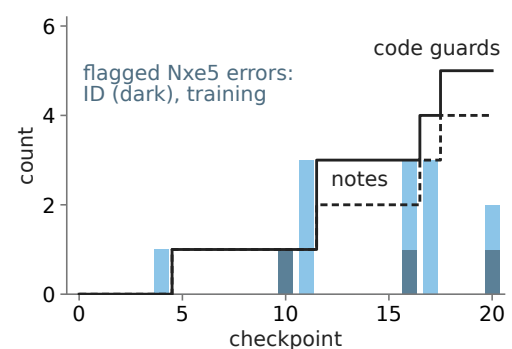
Failure analysis: blunder,
263 cp lost.

After 7...Kxd8



A knight for a pawn down;
mated on move 34.

Anti-Nxe5 rules vs flagged errors



After two bash calls that list and read inventory files, one Python call:

```
$ load_python('/workspace/inventory/tools/chess_agent.py'); print(play_game(board_api))
```

Inside play_game(), the artifact submits the move (sandbox trace):

```
→ "method": "board.submit_move", "params": {"move": "Nxe5"}} ...
{"accepted": true, "submitted_move": "Nxe5", "stockfish_move": "Nxe5", ...
```

checkpoint 20 · strategies/gameplay.md, line 49

Checkpoint-018 repair note: ... As White, do not play `Nxe5` when Black has `...Nc6`, `...d6`, `...Nd7`, or `...Qc7`; ...

tools/chess_agent.py, lines 3576-3580 *the guard, in _early_bad_move_018()*

```
3576 # Open games: Nxe5 is poisoned when Black's c6 knight simply recaptures;
3577 # only the Petrov (...Nf6) book should ever encourage Nxe5.
3578 if san == 'Nxe5' and fullmove <= 8:
3579     if has('p','e5') and (has('n','c6') or has('p','d6') or has('n','d7') or has('q','c7')):
3580         return True
```

tools/chess_agent.py, lines 4299-4301 and 4384-4391 *the opening book returns before the guard filter runs*

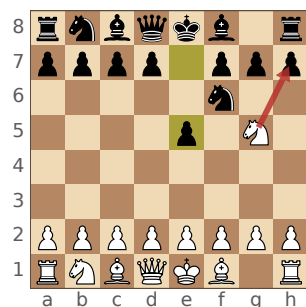
```
4299     if fullmove <= 14 and has('p','e5'):
4300         r = _pick_existing(smap, ['dxe5', 'Nxe5', 'Bxe5', 'exd5', 'd4', 'Nc3', 'Nf3', 'Be2'])
4301         if r: return r
...
4384     book = opening_book_choice(fen,b,c,cast,ep,moves,allowed)
4385     if book: return book
...
4391     good=[m for m in cand if not _early_bad_move_020(b,c,cast,ep,m,fullmove)]
```

Figure 18 GPT-5.5's rules against a known mistake do not stop it. From checkpoint 5 on, GPT-5.5's notes and code checks forbid capturing the e5 pawn early with Nxe5 when the knight can be taken back. Plot: number of such rules per checkpoint (code checks solid, notes dashed). But the opening book (a fixed list of prepared moves) answers before the checks run, and at checkpoint 20 it still picks 6.Nxe5 in this held-out game, a blunder costing 263 cp; the game is lost. Bars: early-Nxe5 mistakes found by our failure analysis in held-out (dark) and training games. The rules pile up while the mistake continues. See Section 3.5 and Appendix E.

GPT-5.6 Luna's notes tell it to play the first legal move listed

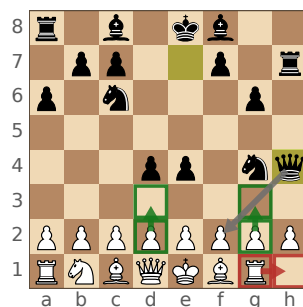
Checkpoint 20, White against skill 5: every move it plays is the first one listed

Before 3.Nxh7



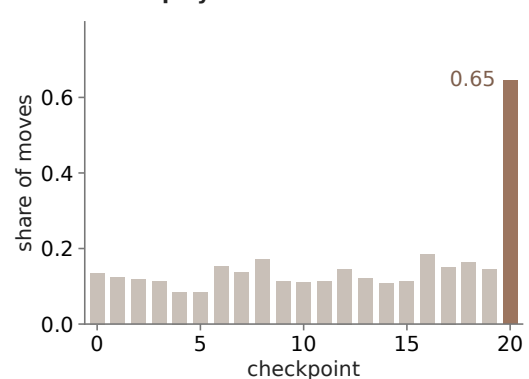
3.Nxh7 is first in the list;
3...Rxh7.

Before 13.Rh1



Only g3 and d3 stop
13...Qxf2#; Rh1 is listed
first.

How often it plays the first move listed



```
$ print(board_api['get_legal_moves']())
→ ['Rh1', 'Nc3', 'Na3', 'h3', 'g3', 'e3', 'd3', 'c3', 'b3', 'a3', 'c4', 'b4', 'a4']
Next turn: 0 reasoning tokens.
$ submit_move {"move": "Rh1"}
→ {"accepted": true, "submitted_move": "Rh1", "stockfish_move": "Qxf2#", ...
    "result": "0-1", "termination": "checkmate"}
```

checkpoint 20 · instructions/turn_budget_watchdog.md, lines 6 and 18

1. ****SYNC (one Python call only):**** execute exactly ``print(board_api['get_legal_moves']())``. The returned list is authoritative. Copy its first item verbatim as ``FALLBACK``. ... Never delay commit to analyze. If the sync list visibly contains a forced check evasion, use it; otherwise use the first listed SAN. Do not compare candidates. The operational objective is a legal completed game, not best play.

strategies/tactical_move_selection.md, line 5

On the SYNC response, copy the first exact legal SAN into ``FALLBACK`` before thinking. ...

Figure 19 GPT-5.6 Luna's notes tell it to play the first legal move listed. To avoid losing games by running out of its 64 model calls per game, the notes at checkpoint 20 tell the agent to copy the first move in the list of legal moves without comparing options. In this held-out game every move is the first one listed, including 3.Nxh7, which loses a knight, and 13.Rh1, which allows checkmate next move. Plot: share of held-out moves that are the first move listed, 0.65 at checkpoint 20 and below 0.2 before. Some checkpoint-20 games still run out of calls. See Section 3.5 and Appendix E.

F Additional Results

F.1 Performance Versus Learning Tokens

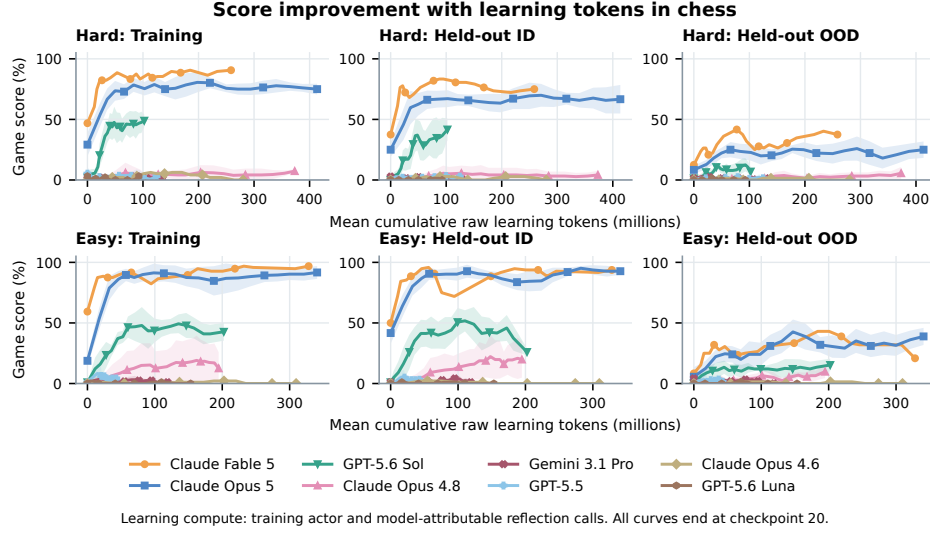


Figure 20 Score improvement with learning tokens in chess. Learning tokens count the model calls spent learning: playing the training games and reflecting on them to write the next checkpoint (held-out games are not counted). Lines are three-checkpoint averages. See Section 3.4 and Appendix F.

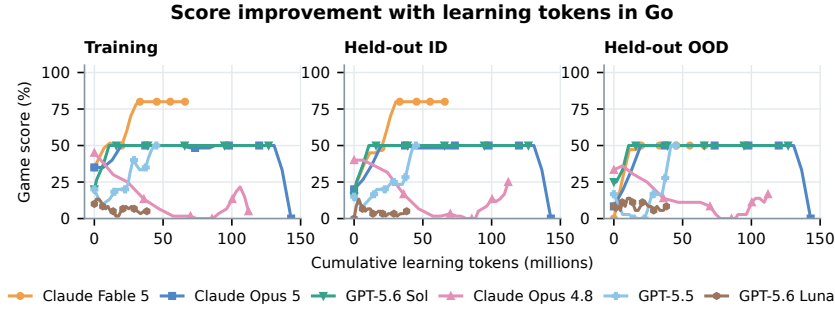


Figure 21 Score improvement with learning tokens in Go. Score against cumulative learning tokens. Lines are three-checkpoint averages. See Section 3.4 and Appendix F.

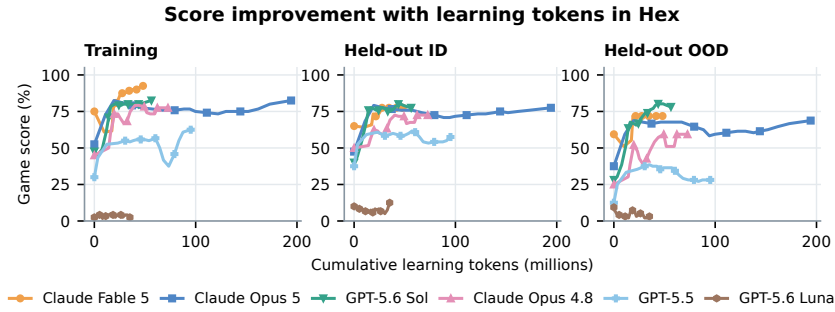


Figure 22 Score improvement with learning tokens in Hex. Score against cumulative learning tokens. Lines are three-checkpoint averages. See Section 3.4 and Appendix F.

F.2 Performance Versus Learning Cost

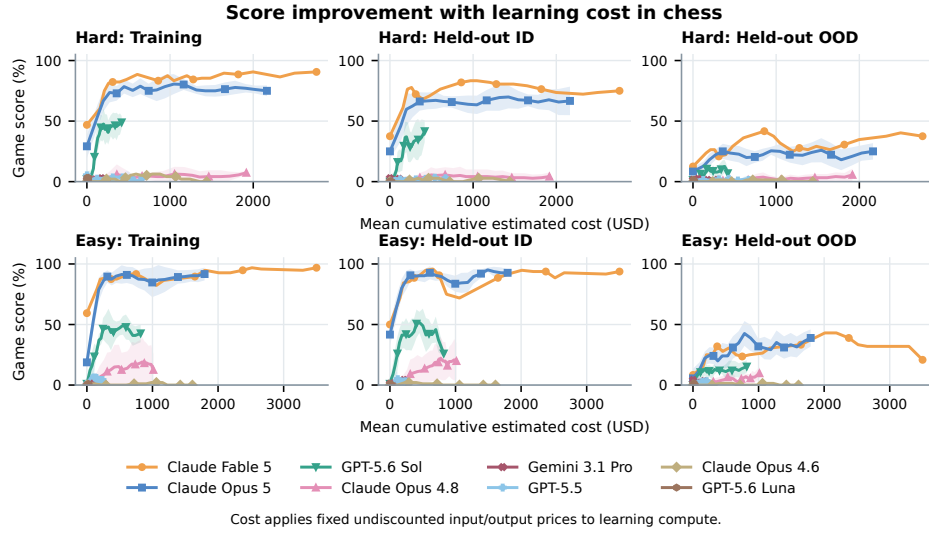


Figure 23 Score improvement with learning cost in chess. Score against cumulative learning cost in dollars, the cost used in P^{sat} (Section 2.2). See Section 3.4 and Appendix F.

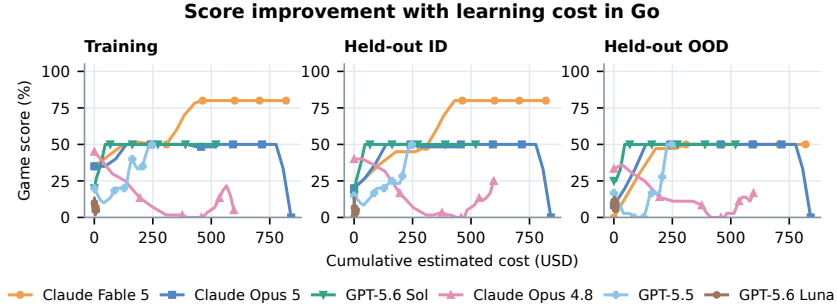


Figure 24 Score improvement with learning cost in Go. Score against cumulative learning cost. Lines are three-checkpoint averages. See Section 3.4 and Appendix F.

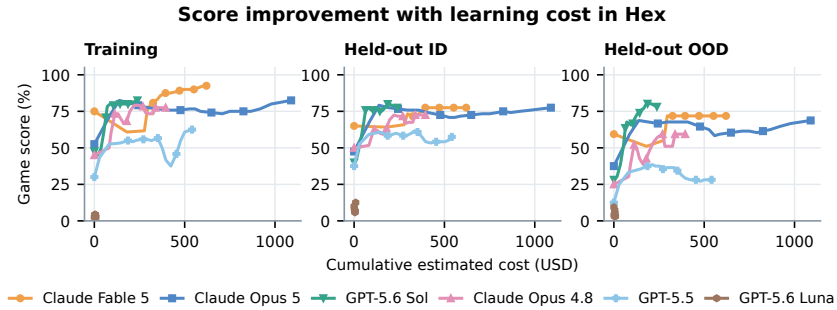


Figure 25 Score improvement with learning cost in Hex. Score against cumulative learning cost. Lines are three-checkpoint averages. See Section 3.4 and Appendix F.

G Failure and Reuse Analysis Procedure

The failure and reuse labels of Section 3.5 are produced offline by Gemini 3.1 Pro, which reads each saved game together with the artifacts available to the agent. Each game passes through the six stages of Table 4. Each failure is then assigned to $\mathcal{F}_{\text{absent}}$ (no artifact covers it), $\mathcal{F}_{\text{missed}}$ (an artifact covers it but was not used), or $\mathcal{F}_{\text{used}}$ (a covering artifact was used and the failure still happened), and a decision counts as reused if any relevant artifact was used.

Stage	Input	What the prompt asks	Output
Failure identification	Game rules; for each decision the position, legal actions, chosen action, and engine evaluation; recorded failures and uncertain engine candidates. No artifacts.	Describe each recorded failure from direct evidence. Label each uncertain candidate; heuristic or unstable engine evidence alone is not a failure. Add failures only when same-decision evidence shows them, at most one optional failure per decision.	Failure list with names, descriptions, and cited events.
Failure verification	Same input plus the draft, marked as an untrusted proposal.	Re-check independently and return the complete corrected list: keep every recorded failure, relabel candidates, keep only directly evidenced additions, and drop duplicates.	Final failure list.
Failure-artifact relation	Verified failures and the records of their decisions; the full contents of every artifact available at that checkpoint; recorded artifact reads and invocations.	Judge whether an artifact covers the failure from its contents, not its name. Judge use separately, from evidence before or during the failure: an invocation and its end event for executables, access and application for text.	Per failure: no covering artifact, or covering artifact used or not used.
Relation verification	Same input plus the draft.	Check every failure against the complete artifact list, including artifacts the draft omitted. Producing the failing action does not by itself make an artifact covering.	Final coverage and use judgment.
Decision-level reuse	Every game-play decision, in batches of up to 20, with its verified failures and relation judgments; artifacts; recorded reads and invocations.	For each decision and artifact, judge relevance to choosing or executing the action, use from same-decision evidence, and outcome. An artifact that covers a failure at the decision is relevant.	Per pair: relevant, used, and successful or failure remained.
Reuse verification	Same input plus the draft.	Re-audit and return complete corrected rows. Availability, file listings, names, automatic loading, and evidence from other decisions do not establish use.	Final reuse judgments.

Table 4 Analyzer stages in the order they run. Each call goes to Gemini 3.1 Pro, which is told to treat the records as data, use only the given evidence and game rules, and return one JSON object. A claim that an artifact was used is kept only if the logs show it being run or read. Requests also include engine evaluations that the agents never saw. Coverage: training and held-out ID games at checkpoints 0–20.

H Failure and Reuse Diagnostics Across Games

The figures below give additional failure and reuse diagnostics for chess, Go, and Hex. Matching views are placed consecutively across games.

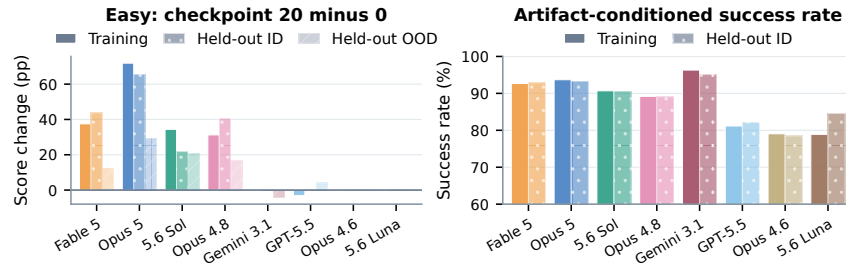


Figure 26 Score change and success after reuse in the Easy chess condition. Same as Figure 6, which shows the Hard condition. See Section 3.5 and Appendix H.

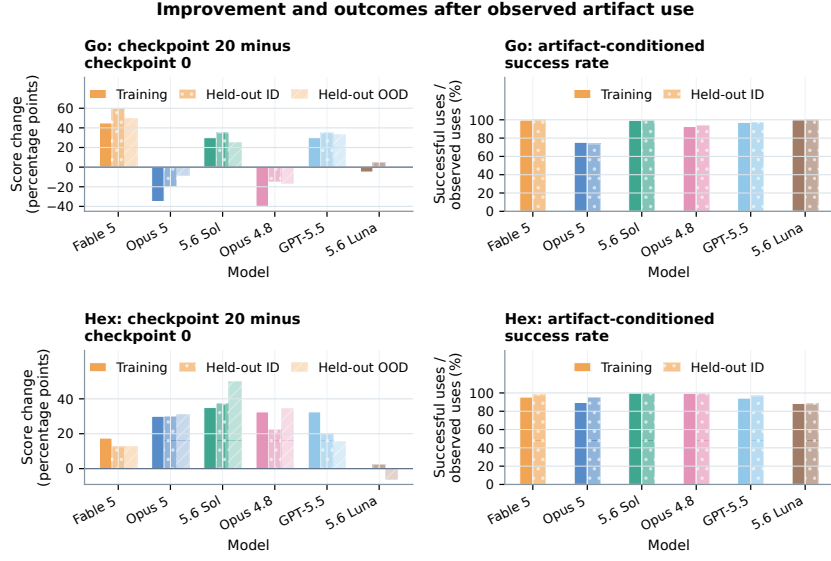


Figure 27 Improvement and outcomes after artifact use in Go and Hex. Left: checkpoint-20 score minus checkpoint-0 score in each evaluation split. Right: artifact-conditioned success rate, i.e., how often a decision succeeds when an artifact was used (an association, not a causal effect). See Section 3.5 and Appendix H.

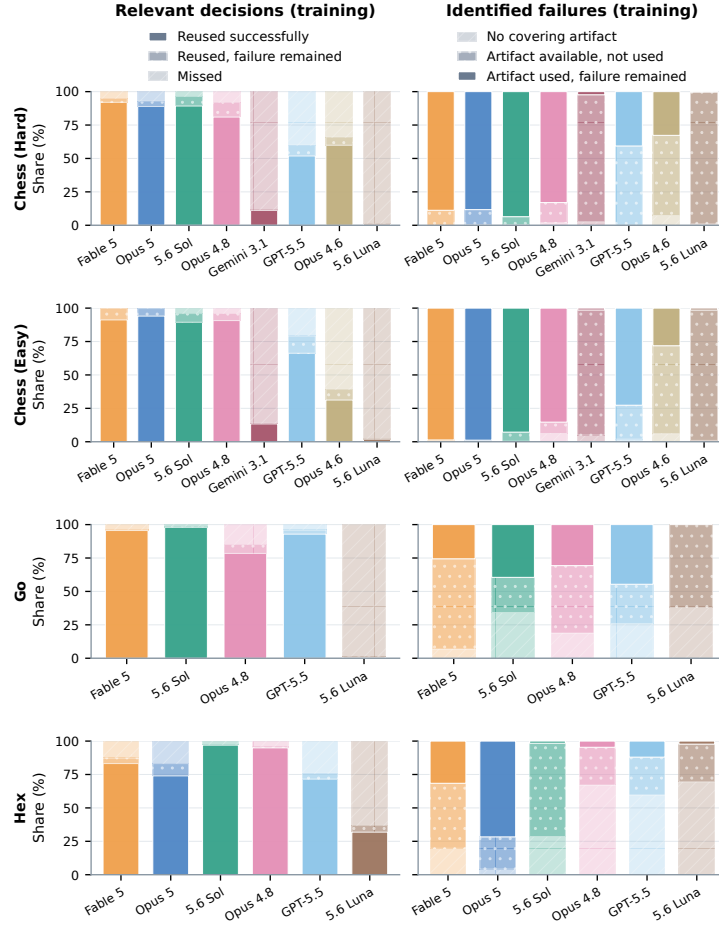


Figure 28 Reuse and failure breakdown on training games. Same as Figures 4 and 29, which use held-out ID games. See Section 3.5 and Appendix H.

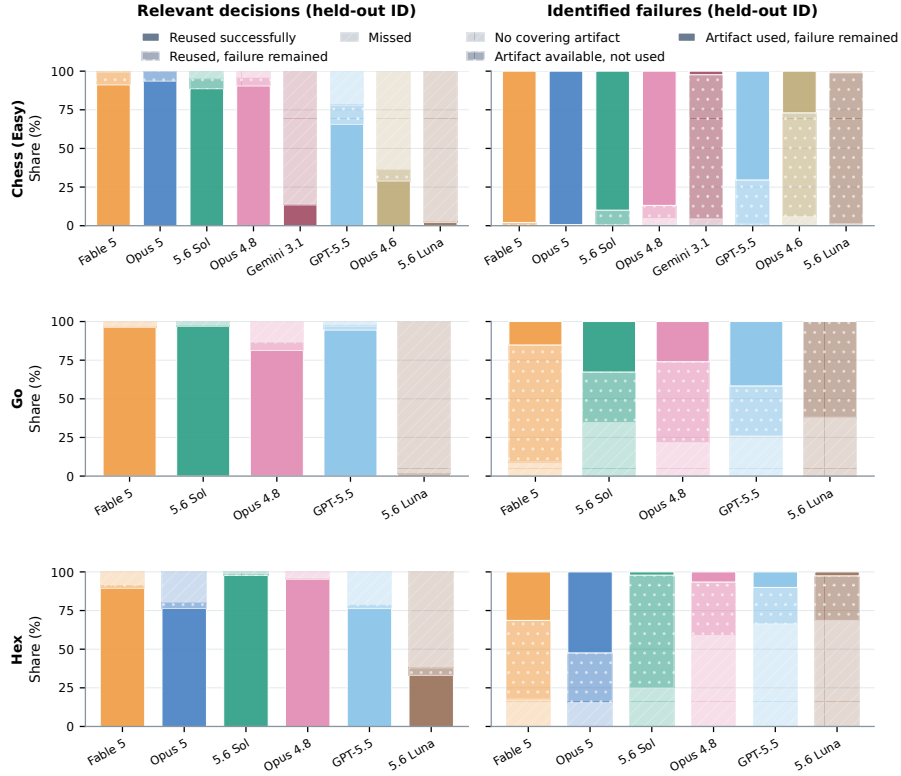


Figure 29 Reuse and failure breakdown in the Easy chess condition, Go, and Hex. Held-out ID games; same as Figure 4, which shows the Hard chess condition. See Section 3.5 and Appendix H.

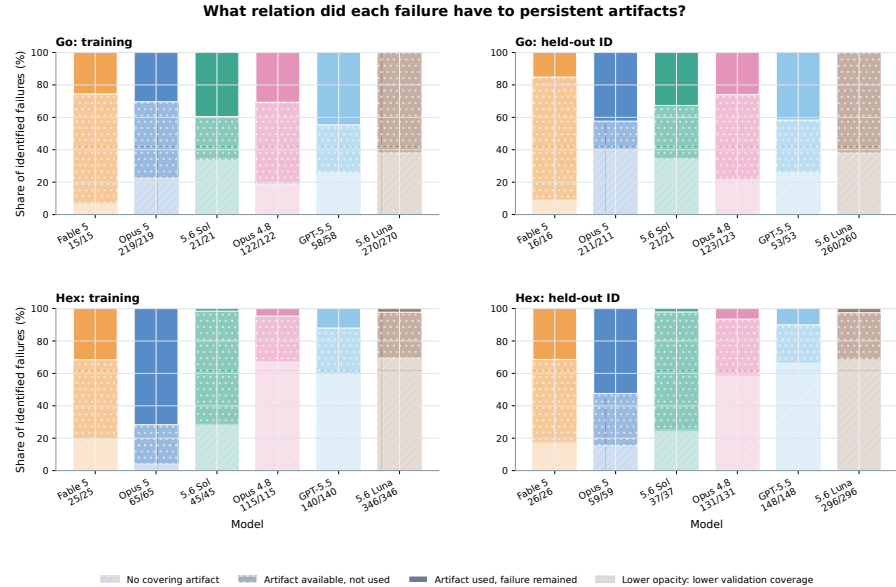


Figure 30 Relation between identified failures and persistent artifacts in Go and Hex. Each verified failure contributes once: no covering artifact existed, a covering artifact existed but was not used, or an artifact was used but the failure remained. Fractions are shown within each model, game, and split; counts beneath labels show verified/identified failures. See Section 3.5 and Appendix H.

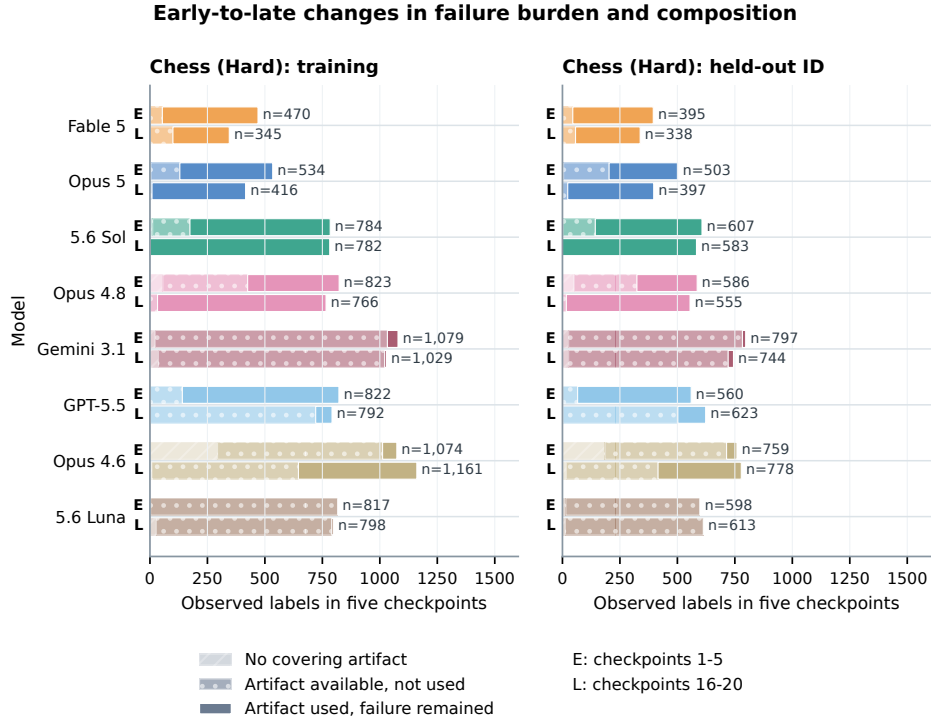


Figure 31 How chess failures change from early to late checkpoints. “E” sums checkpoints 1–5 and “L” checkpoints 16–20 (80 training and 60 held-out ID games per lineage in each). Claude Fable 5 and Claude Opus 5 make fewer failures late. Late failures are mostly “artifact available but unused” for Gemini 3.1 Pro and GPT-5.6 Luna, and mostly “failed despite use” for Claude Opus 5, Claude Opus 4.8, and GPT-5.6 Sol; GPT-5.5 moves toward non-use. These are descriptive, not causal. See Section 3.5 and Appendix H.

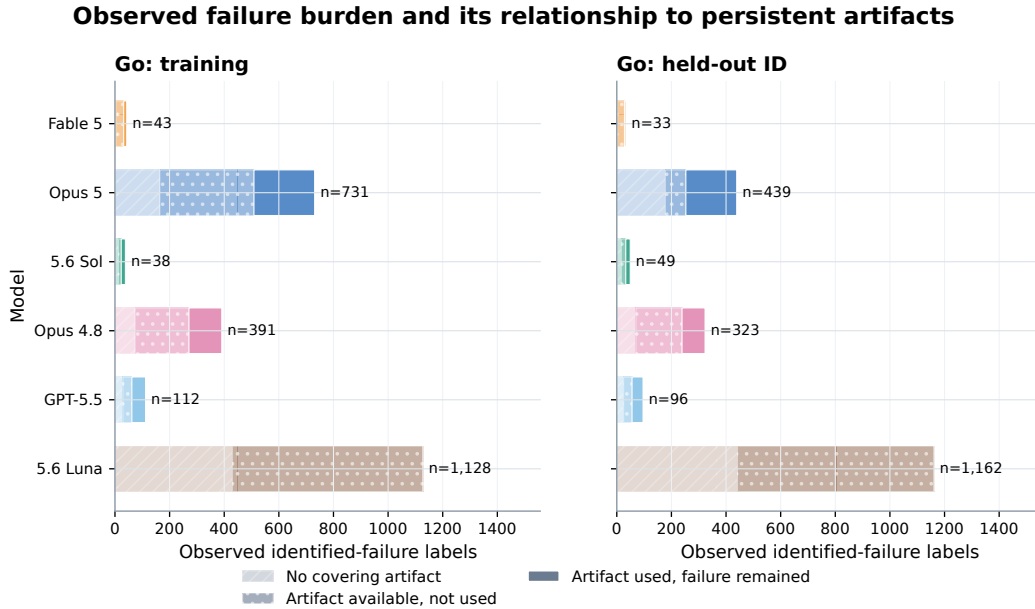


Figure 32 Number of Go failures and their relation to artifacts. Bar length is the number of failures over checkpoints 1–20, split into the three classes (no covering artifact, artifact not used, failed despite use); n is the number of failures. See Section 3.5 and Appendix H.

Observed failure burden and its relationship to persistent artifacts

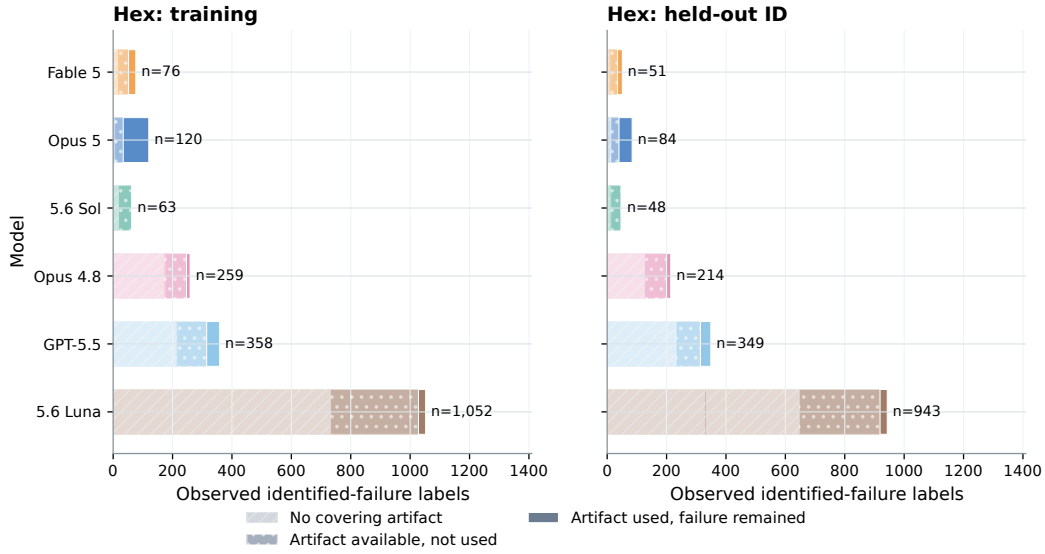


Figure 33 Number of Hex failures and their relation to artifacts. Same as Figure 32. Counts can be compared within a game, not across games. See Section 3.5 and Appendix H.

Artifact coverage and reuse in chess training games

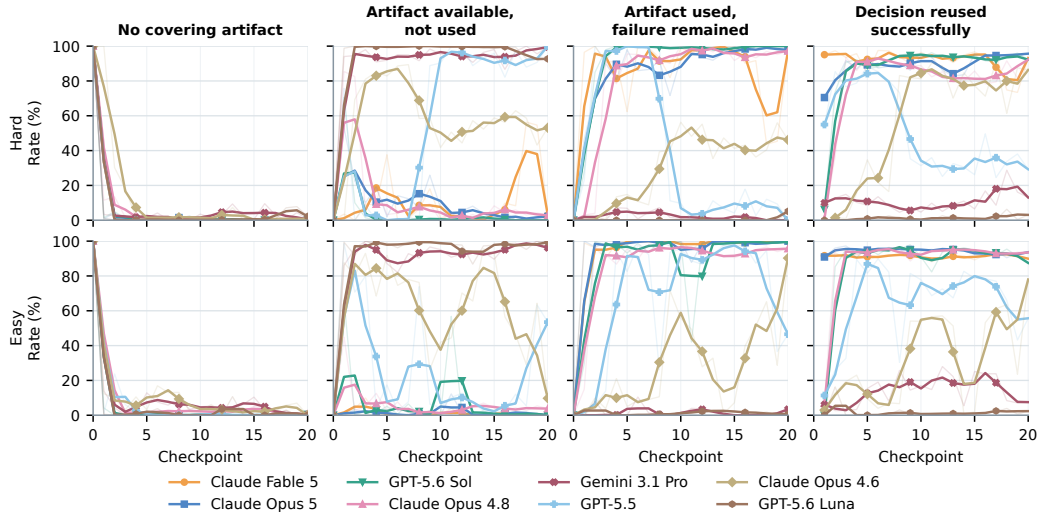


Figure 34 Artifact coverage and reuse at each checkpoint, chess training games. The first three panels show, per failure, how often no artifact covered it, a covering artifact existed but was not used, or it happened even though an artifact was used. The last panel shows the share of relevant decisions (moves where at least one saved artifact is relevant to choosing or playing the move) where an artifact was used successfully. Faint markers are single checkpoints; lines are three-checkpoint averages. See Section 3.5 and Appendix H.

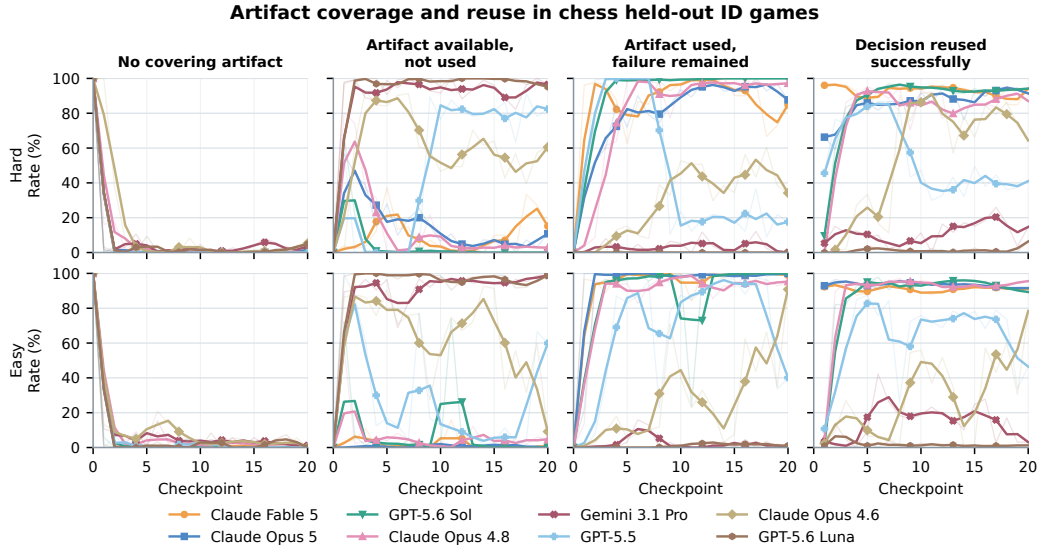


Figure 35 Artifact coverage and reuse at each checkpoint, chess held-out ID games. Same as Figure 34. The model never reviews these games, so they show how the artifacts carry over to unseen games. See Section 3.5 and Appendix H.

Composition of artifact-reuse outcomes across checkpoint phases

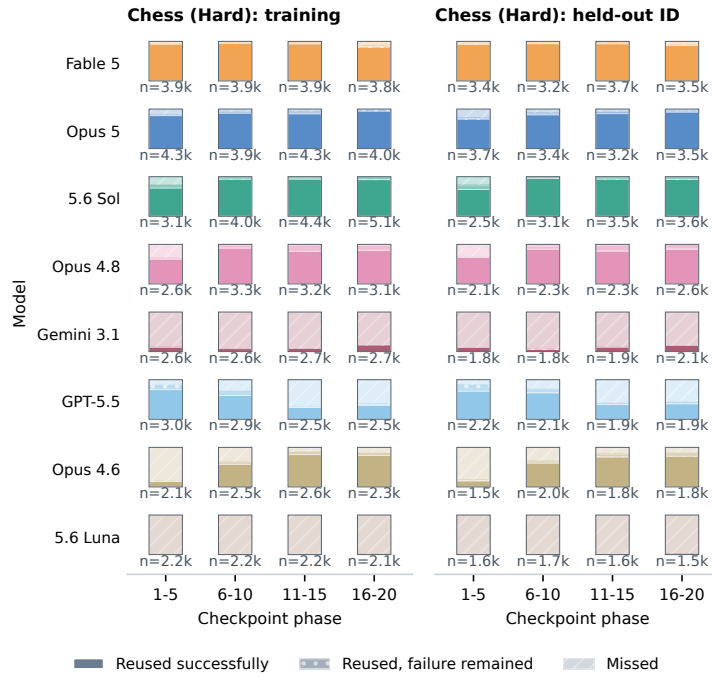


Figure 36 Chess reuse outcomes over time. For each model, four bars for checkpoints 1–5, 6–10, 11–15, and 16–20. Each relevant decision (a move where at least one saved artifact is relevant to choosing or playing it) counts once: reused successfully, reused but the failure remained, or missed (a relevant artifact was not used); n is the number of relevant decisions. Summed over the four phases, the totals match Figures 4 and 28. See Section 3.5 and Appendix H.

Composition of artifact-reuse outcomes across checkpoint phases

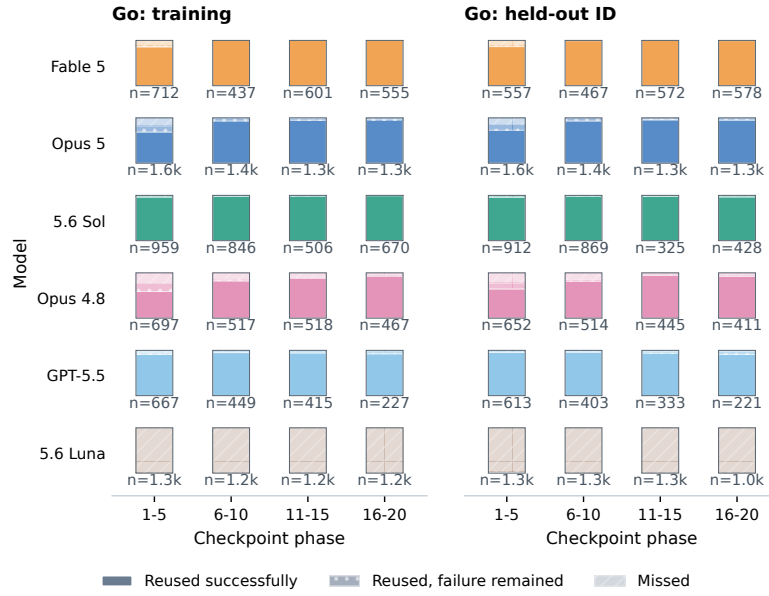


Figure 37 Go reuse outcomes over time. For each model, bars run from checkpoints 1–5 to 16–20. Each relevant decision (a move where at least one saved artifact is relevant to choosing or playing it) counts once: reused successfully, reused but the failure remained, or missed; n is the number of relevant decisions. See Section 3.5 and Appendix H.

Composition of artifact-reuse outcomes across checkpoint phases

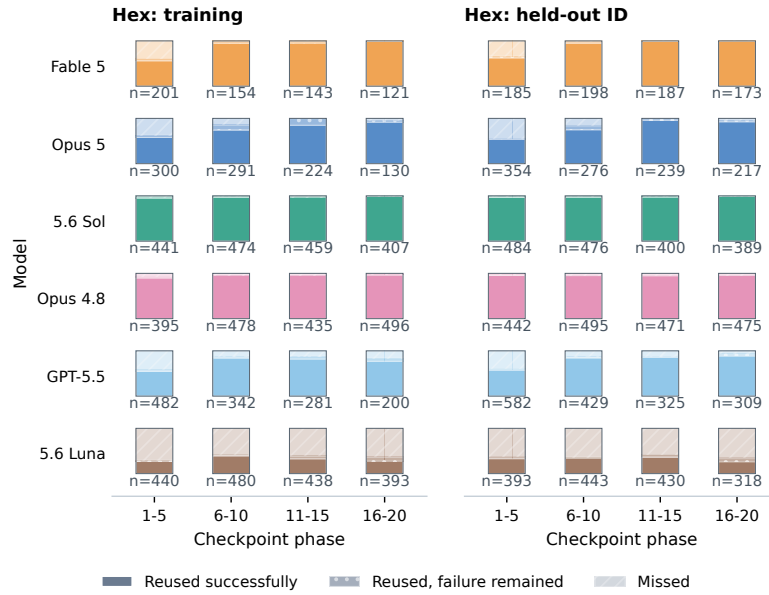


Figure 38 Hex reuse outcomes over time. Same as Figure 37. See Section 3.5 and Appendix H.

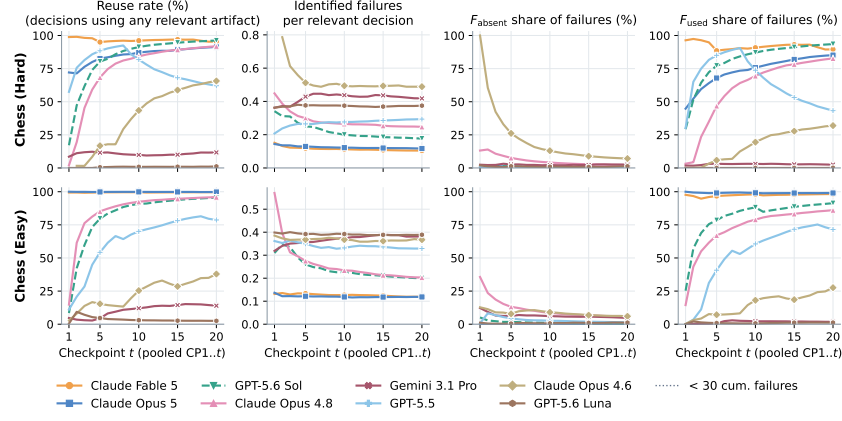


Figure 39 Cumulative improvement-loop metrics in chess, all eight models. Same as Figure 7, for the Hard (top) and Easy (bottom) chess conditions. Each point pools training and held-out ID decisions from checkpoint 1 to t . Columns: reuse rate; failures per relevant decision (a move where at least one saved artifact is relevant to choosing or playing it); F_{absent} ; and F_{used} . Dotted: fewer than 30 failures so far. See Section 3.5 and Appendix H.

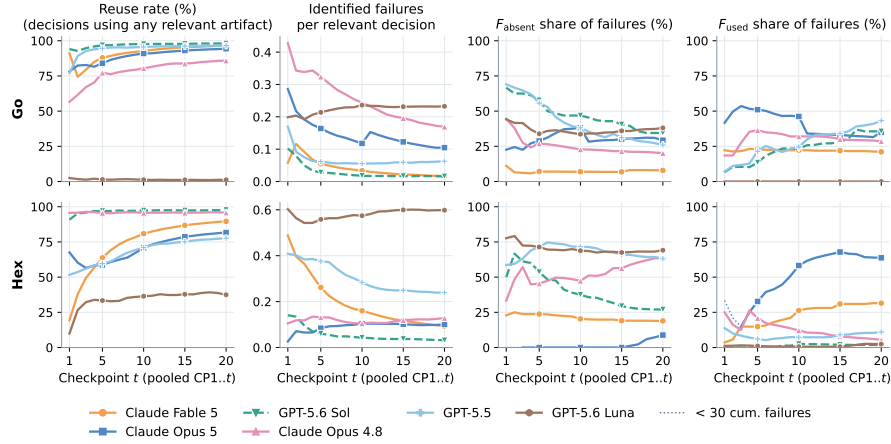


Figure 40 Cumulative improvement-loop metrics in Go and Hex. Same as Figure 7; each point pools training and held-out ID decisions from checkpoint 1 to t . Claude Opus 4.6 and Gemini 3.1 Pro are analyzed in chess only. Dotted: fewer than 30 failures so far. See Section 3.5 and Appendix H.

I Actor and Reflection Prompts

Every actor and reflection call receives the system prompt below, then the game’s actor prompt or the reflection prompt, plus a one-line budget message. The texts are from the harness, with file paths and the validator command slightly simplified; {...} marks a value filled in at run time and [...] an omission. The prompts are the same for all models (apart from the tool-call fallback at the end) and for both chess conditions; Go and Hex change only the rules, the game API, the call budget, and the tool time limit.

I.1 System prompt (all games, actor and reflection)

You interact with an environment through a minimal API. Improve your score over repeated epochs by building reliable infrastructure from your interactions, feedback, rewards, and failures.

You can create and improve tools such as Python functions, maintain persistent memory, define strategies for using tools and memory, and delete inventory that is no longer useful. Decide what to create, use, revise, retain, or delete.

You have Bash and Python inside a constrained sandbox. Bash can inspect your persistent inventory and manage files in permitted writable directories. Python provides the environment API and can execute tools you create. Python imports and direct filesystem access are unavailable; use Bash and `load_python(path)` for permitted files.

I.2 Actor prompts

The actor has a `bash` tool (the inventory is read-only and `/session` is writable), a persistent `python` interpreter that exposes the game API and the library’s tools, and a native move-submission tool (`submit_move` in chess, `submit_action` in Go and Hex). `{color}` and `{actor_role}` are the actor’s side.

Chess (Hard and Easy).

A new game has started. You are playing `{color}`. Persistent infrastructure is available read-only under `/workspace/inventory`. `/session` is writable and lasts only for this game. Use Bash, Python, existing infrastructure, newly created temporary tools, or direct reasoning as you decide. You have at most 64 model calls for this game.

Python exposes `board_api` as a dictionary with `get_fen()`, `get_legal_moves()`, `make_move(move)`, `undo_move()`, `turn()`, and `submit_move(move)`, for example `board_api["get_legal_moves"]()`. `board_api["submit_move"](move)` commits one move in Standard Algebraic Notation and returns Stockfish’s reply and the new position. The native `submit_move` tool performs the same action. Submit legal moves to maximize your score.

Persistent failure-mode and artifact indexes are available under `/workspace/inventory/tracking`.

Go.

A new go game has started. You are playing `{actor_role}`. Persistent infrastructure is available read-only under `/workspace/inventory`. `/session` is writable and lasts only for this game. Use Bash, Python, existing infrastructure, newly created temporary tools, or direct reasoning as you decide. You have at most 32 model calls for this game.

Black uses X and White uses O. Actions are coordinates such as C4 or pass. Captures, no-suicide, positional superko, two-pass termination, area scoring, and the displayed komi apply. Final outcomes use Chinese rules with dead-stone adjudication by the authoritative scorer. The action cap is 75 total actions. If that cap is reached before two passes, the scorer adjudicates the current position.

Python exposes `game_api` as a dictionary with `get_state()`, `get_legal_actions()`, `apply_action(action)`, `undo_action()`, `current_player()`, and `submit_action(action)`. `game_api["submit_action"](action)` commits one legal action, lets the automated opponent respond when the game continues, and returns the new state. The native `submit_action` tool performs the same action. Each Bash or Python call may run for at most 240 seconds. Submit legal actions to maximize your score.

Persistent failure-mode and artifact indexes are available under `/workspace/inventory/tracking`.

Hex. The Hex prompt matches the Go prompt except for the game name and the rules paragraph.

A new hex game has started. You are playing {actor_role}. [...]

Red uses X and connects north to south. Blue uses O and connects west to east. Actions are coordinates such as c4. The first completed connection wins.

[...]

Per-call budget message (actor). This message is appended before every actor model call (64 calls in chess, 32 in Go and Hex).

Game model-call budget: {used}/{maximum} calls used; {remaining} remain.

Tool-timeout recovery (Go and Hex). If a Bash or Python call exceeds the tool timeout, the actor gets one final call with only `get_state`, `get_legal_actions`, and `submit_action` available. This message follows the budget message.

Your Bash or Python tool timed out. This is your only recovery model call. You may only inspect the current state, inspect legal actions, and submit one legal action with the provided native tools. Bash, Python, and further search are unavailable. Submit a legal action now or the game is forfeited.

1.3 Reflection prompt

Reflection has a single `bash` tool (“Run Bash inside the writable transactional `/workspace`.”). The prompt below uses the chess values; Go and Hex use 300.0 instead of 1800.0 seconds. `/workspace/feedback` contains only the *training* games of the current checkpoint (moves, engine annotations in chess, traces, session files), a score summary, and the protocol files (`SCHEMA.md`, the validator, and the parent inventory). Held-out games are never copied there.

Game transcripts, annotations, failures, per-game scratch files, mutation feedback, and your current infrastructure are available under `/workspace`. Persistent infrastructure is under `/workspace/inventory`. On a retry, prior reflection logs are available under `/workspace/feedback/reflection_attempts`. Improve the infrastructure for future games.

You can create or modify Python tools, create or modify persistent memory, create or modify strategies for using tools and memory, and delete inventory that is no longer useful. Use the Bash tool to inspect and edit `/workspace/inventory`. Invoke Bash directly rather than describing or printing a tool call. Each Bash call may run for at most 1800.0 seconds. You have at most 50 reflection model calls and will be told how many remain. Finish early when the work is complete. The inventory is validated after the final call.

Maintain `failure_modes.json` and `artifacts.json` under `/workspace/inventory/tracking` while inspecting the canonical training trajectories under `/workspace/feedback`. Reuse a failure-mode ID when the causal problem is the same; create a new ID only for a materially different problem. Map failures to exact training game IDs. A trajectory need not be mapped when you find no failure. Preserve mitigated and merged modes rather than deleting them.

Maintain stable artifact IDs for persistent tools, memory, strategies, instructions, and other infrastructure. Keep an ID when revising the same capability, but create a new ID for a materially different capability rather than repurposing an old ID. Record paths, executable entrypoints, addressed failure modes, and active or retired status. Every functional inventory file you create, modify, or retire must belong to an artifact addressing at least one failure mode. Retired artifacts must not leave implementation files in the active inventory.

Use these exact JSON shapes. 'failure_modes.json' is '{ "schema_version": "failure_modes.v1", "failure_modes": [{ "id": "FM-0001", "description": "...", "status": "active", "trajectory_ids": ["checkpoint_000_train_s1_white"] }] }'. Status is 'active', 'mitigated', or 'merged'; only a merged entry adds 'merged_into': 'FM-...'. 'artifacts.json' is '{ "schema_version": "artifacts.v1", "artifacts": [{ "id": "AR-0001", "description": "...", "kind": "tool", "paths": ["tools/example.py"], "entrypoints": ["tools/example.py:choose_move"], "failure_mode_ids": ["FM-0001"], "status": "active" }] }'. Kind is 'tool', 'memory', 'strategy', 'instruction', or 'other'; status is 'active' or 'retired'; non-executable artifacts use an empty 'entrypoints' list. The same schemas and transition rules are in /workspace/feedback/protocol/SCHEMA.md.

Before finishing, run the protocol validator /workspace/feedback/protocol/validate_tracking.py and correct structural errors before the final response.

Per-call budget message (reflection).

Reflection budget: {used}/{maximum} calls used; {remaining} remain.

Final reflection call. This replaces the budget message on the last permitted call.

Reflection budget: {used}/{maximum} calls used; {remaining} remains. Final reflection call {turn}/{maximum}. Given the evidence so far, make the best persistent infrastructure edits now. Invoke the Bash tool directly for edits under /workspace/inventory rather than describing or printing a tool call. The inventory is validated after this call.

Synchronize failure_modes.json and artifacts.json with the final inventory, then run the protocol validator before finishing.

Retry addendum. If a reflection attempt is rejected (for example, the new library fails to load), the harness starts a new attempt with this text appended (up to 6 attempts in chess, 3 in Go and Hex). {latest_error} holds the rejection message; a rendered example from a Chess (Hard) run follows the template.

This is a retry after a rejected reflection attempt. Before editing, read /workspace/feedback/reflection_retry_history.json and the prior logs under /workspace/feedback/reflection_attempts. Resolve the recorded validation error instead of repeating the rejected change.

Latest rejection: {latest_error}

Rendered example of the last line:

Latest rejection: ValueError: candidate infrastructure failed smoke validation: persistent tool load failed: NameError: name 'object' is not defined

Tool-call fallback. Most Go and Hex runs of models other than Claude Fable 5 go through an inference gateway, which appends this sentence to the system prompt.

Use structured tool calling whenever you call a tool. If structured tool calling is unavailable, end your response with FINAL_TOOL_CALL_JSON: on its own line, followed by exactly one JSON object in the form {"functionCall":{"name":"tool_name","args":{}}}. Anything after that marker is treated as a real tool call, so never use the marker in examples.